

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

**«Вятский государственный университет»
(ФГБОУ ВО «ВятГУ»)**

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Допущено к защите
Руководитель проекта
_____/Сергеева Е.Г./
«__» _____ 2026 г.

Разработка однопользовательской 2D игры «Сирра»

Пояснительная записка курсового проекта по дисциплине

«Разработка мультимедийных приложений»

ТПЖА.090301.957 ПЗ

Разработал студент группы ИВТб-2302

_____/Матешко В. В./

Руководитель

_____/Сергеева Е.Г./

Консультант

_____/Сергеева Е.Г./

Проект защищен с оценкой «_____» _____
(оценка) (дата)

Члены комиссии	_____ / _____/
	(подпись) (ФИО)
	_____ / _____/
	(подпись) (ФИО)
	_____ / _____/
	(подпись) (ФИО)

Киров 2026

Реферат

Матешко В.В. Разработка однопользовательской 2D игры «Сурра». ТПЖА.090301.957 ПЗ: Курс. проект / ВятГУ, каф. ЭВМ; рук. Сергеева Е.Г. - Киров, 2026. – ПЗ 102 с., 32 рис., 2 прил.

2D-ИГРА, РИТМ-ИГРА, РАЗРАБОТКА ИГРЫ, ВИЗУАЛЬНАЯ НОВЕЛЛА, СЮЖЕТНОЕ ВЕТВЛЕНИЕ, СИСТЕМА ОЦЕНКИ, ПАРСИНГ SM-ФАЙЛОВ, ОБРАБОТКА ДЛИННЫХ НОТ, СИНХРОНИЗАЦИЯ АУДИО И ГЕЙМПЛЕЯ, GODOT, GDSCRIPT.

Объект курсового проекта – однопользовательские 2D-игры, музыкальный элемент как ключевая игровая механика, интерактивное повествование, переходы между игровыми состояниями.

Предмет курсового проекта – однопользовательская 2D-игра «Сурра», сочетающая визуальную новеллу и ритм-механику приготовления кофе, а также проектирование и реализация соответствующих алгоритмов.

Цель курсового проекта – разработка и реализация однопользовательской 2D-игры «Сурра» на игровом движке Godot, в которой точность прохождения ритм-сцен влияет на повествование развития истории через диалоги и итоговые концовки.

Результатом выполнения курсового проекта является функциональный прототип игры «Сурра», включающий главное меню, пролог, диалоговые сцены, ритм-модуль с обработкой обычных и длинных нот, систему оценки попаданий, экран концовки и переходы между всеми ключевыми состояниями.

Содержание

Введение	5
-----------------	----------

1 Анализ предметной области	7
------------------------------------	----------

1.1	Обзор предметной области	7
1.2	Анализ аналогов	8
1.2.1	Анализ игры «VA-11 Hall-A»	8
1.2.2	Анализ игры «Coffee Talk»	9
1.2.3	Анализ игры «Yunyun Syndrome!? Rhythm Psychosis»	10
1.2.4	Анализ игры «Glisynth»	11
1.3	Требования к разработке	12
1.3.1	Функциональные требования	13
1.3.2	Нефункциональные требования	13
1.4	Постановка задач	14
1.5	Заключение	15

2 Проектирование	16
-------------------------	-----------

2.1	Разработка архитектуры и структуры игры	18
2.2	Разработка состояний игры	20
2.3	Проектирование основных алгоритмов	21
2.3.1	Проектирование главного меню и настроек	21
2.3.2	Проектирование сюжетных сцен новеллы	22
2.4	Проектирование уровней ритм-игры	22
2.5	Проектирование алгоритма обработки файла нот	25
2.6	Проектирование алгоритма ритм-игры	27
2.7	Проектирование обычных и длинных нот	29
2.8	Проектирование алгоритма оценки действий игрока	30
2.9	Проектирование алгоритма выбора концовки	32

Подп. и дата		Инв. № дубл.		Взам. инв. №		Подп. и дата		ТПЖА.090301.957 ПЗ				
Инв. № подл.	Изм	Лист	№ докум.	Подп.	Дата	Разработка однопользовательской 2D игры «Супра»			Лит.	Лист	Листов	
	Разраб.	Матешко								3	102	
	Пров.	Сергеева										
	Н. контр.											
	Утв.											

2.10	Заключение	34
3	Реализация	35
3.1	Выбор инструментария и настройка проекта	35
3.2	Построение структуры игрового процесса игры	35
3.3	Реализация игрового движка	37
3.3.1	Создание игровых ресурсов	37
3.3.2	Реализация главного меню	38
3.3.3	Реализация настроек	38
3.3.4	Реализация паузы	39
3.3.5	Реализация пролога	40
3.3.6	Реализация новеллы	41
3.3.7	Реализация ритм-игры	44
3.3.8	Реализация концовок новеллы	47
3.4	Реализация управления в ритм-игре	48
3.5	Эволюция интерфейса и визуального решения	48
3.6	Экспериментальная апробация	50

	Заключение	52
--	-------------------	-----------

	Список литературы	53
--	--------------------------	-----------

	Приложение А. Экранные формы	56
--	-------------------------------------	-----------

	Приложение Б. Листинги исходного кода программных модулей	59
--	--	-----------

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ				Лист
Изм.	Лист	№ докум.	Подп.	Дата					4

Введение

В современном обществе развлечения являются важной частью досуга [1]. Одним из направлений этой сферы является индустрия видеоигр. Развитие цифровых технологий расширило возможности создания игр, тем самым сделав формат игр вариативнее.

В современной индустрии развлечений видеоигры занимают отдельное место [1][2]. Пользователям интересны как AAA-проекты, так и инди-игры, разрабатываемые как одиночными разработчиками, так и командами энтузиастов [2][3].

Разработчики в этой отрасли стремятся предложить игрокам новые форматы взаимодействия, оригинальные истории, дизайн и уникальные механики, которые впоследствии могут привести к появлению нового жанра [5].

В жанре ритм-игр основой игрового процесса является попадание по «нотам» – движущимся объектам – в ритм музыки. Такой формат задает активное взаимодействие пользователя с игрой.

Таким образом, у меня возникла идея создать собственный проект в жанре ритм-игр с акцентом на атмосферу и повествование.

Однако я преследую цель создать игру, в которой уютная внешняя атмосфера сочетается с мрачным вампирским миром и напряженным повествованием. Чтобы реализовать такой контраст, я пришла к идее разработать визуальную новеллу про вампирскую кофейню, где протагонист работает бариста. В истории поднимаются темы внутреннего конфликта персонажей и последствий прохождения, включая неблагоприятные концовки. В игровой процесс встроен элемент ритм-игры, отвечающий за приготовление кофе.

Цель проекта: разработать оригинальную однопользовательскую 2D-игру, в которой присутствует взаимодействие с ритмом музыки и предусмотрен линейный сюжет с ограниченным числом ветвлений. В ходе игры пользователь будет общаться с вампирами и готовить кофе клиентам, играя в ритм-игру, тем самым зарабатывая очки, от которых зависит развитие истории и концовка игры – будет ли существовать вампирская кофейня или нет.

Задачи проекта:

а) Проанализировать существующие проекты и выделить механики, применимые в рамках выбранных жанров;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						Лист
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ					5

- б) Сформировать требования к проекту;
- в) Спроектировать архитектуру игры, определив алгоритмы поведения и структуру состояний;
- г) Разработать базовые модули и реализовать их с использованием игрового движка Godot;
- д) Провести тестирование, проанализировать результаты и сформулировать направления дальнейшего развития.

Объект исследования: однопользовательские 2D-игры, визуальные новеллы, ритм-игры, игровые системы интерактивного повествования и переходы между игровыми состояниями.

Предмет исследования: однопользовательская 2D-игра «Сирра», сочетающая визуальную новеллу и ритм-механику приготовления кофе, а также алгоритмы обработки нот, оценки действий игрока, отображения сюжетных сцен, переходов между игровыми состояниями и формирования сюжетных концовок.

Актуальность проекта в рамках данной работы обусловлена сочетанием следующих факторов:

- а) Выбор видеоигр как одного из форматов досуга [1];
- б) Значимая роль музыкального сопровождения в формировании атмосферы игрового процесса [4];
- в) Применение сюжетного подхода, в котором спокойная внешняя обстановка сочетается с более напряженным повествованием [5];
- г) Тематическая выразительность кофейной эстетики в сочетании с вампирским сеттингом [9][10];
- д) Интерес к совмещению жанров как к способу формирования более цельного игрового опыта [5][11][12].

Инов. № подл.	Подп. и дата
Взам. инв. №	Инов. № дубл.
Подп. и дата	
Инов. № подл.	

Изм.	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Лист
6

1 Анализ предметной области

Ритм-игры – это жанр видеоигр, в основе которого лежит своевременное выполнение действий в соответствии с ритмом музыкального сопровождения. Точность действий влияет на набор очков, качество прохождения и итоговый результат.

Визуальная новелла – это жанр видеоигр, игровой процесс которого строится на чтении текста, выборе реплик или действий, влияющих на развитие истории и ее концовку.

1.1 Обзор предметной области

Игровой процесс ритм-игр требует от пользователя концентрации, реакции и чувства ритма, но при этом сохраняет понятную базовую механику взаимодействия.

Вместе с тем во многих проектах данного жанра основной акцент делается на механике, тогда как сюжетная часть и атмосфера получают меньше внимания, либо вовсе отсутствуют [6][7][8]. В результате игровой опыт нередко ограничивается только серией действий под музыку [6][7][8].

В рамках проекта «Сирра» эта проблема решается через объединение ритм-игры и визуальной новеллы. Ритм-механика поддерживает динамичный темп, а повествовательная часть добавляет персонажей, диалоги и развитие истории.

В рамках проекта музыкальные сцены встроены в контекст событий и связаны с развитием повествования. Это реализуется через концепцию вампирской кофейни, где игрок выступает в роли бариста, общается с посетителями и готовит напитки с использованием ритм-элементов.

Таким образом, проект направлен на создание целостного игрового опыта, сочетающего два жанра: ритм-игру и визуальную новеллу.

Инв. № подл.	Подп. и дата				ТПЖА.090301.957 ПЗ	Лист	
	Инв. № дубл.					7	
	Взам. инв. №						
	Подп. и дата						
Изм.	Лист	№ докум.	Подп.	Дата			

СВЯЗИ.	Вместе с тем во многих проектах данного жанра основной акцент делается на механике, тогда как сюжетная часть и атмосфера получают меньше внимания, либо вовсе отсутствуют [6][7][8]. В результате игровой опыт нередко ограничивается только серией действий под музыку [6][7][8]. В рамках проекта «Сурра» эта проблема решается через объединение ритм-игры и визуальной новеллы. Ритм-механика поддерживает динамичный темп, а повествовательная часть добавляет персонажей, диалоги и развитие истории. В рамках проекта музыкальные сцены встроены в контекст событий и связаны с развитием повествования. Это реализуется через концепцию вампирской кофейни, где игрок выступает в роли бариста, общается с посетителями и готовит напитки с использованием ритм-элементов. Таким образом, проект направлен на создание целостного игрового опыта, сочетающего два жанра: ритм-игру и визуальную новеллу.
--------	--

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

В рамках данного исследования рассматриваются игры, обладающие преимущественно следующими признаками:

- а) Наличие главного героя, выступающего в роли обслуживающего персонала (например, бариста, официанта или бармена);
- б) Визуальное исполнение в 2D-стиле;
- в) Принадлежность к жанру визуальной новеллы, ритм-игры или их сочетанию;
- г) Акцент на взаимодействие с персонажами, атмосферу и игровые действия, влияющие на восприятие истории.

Игра «VA-11 Hall-A» представляет собой визуальную новеллу, в которой игрок выступает в роли бармена в киберпанк-баре. Основной игровой процесс строится вокруг общения с посетителями и приготовления напитков, от которых зависит развитие диалогов и состояние персонажей.

Проект выбран как аналог, поскольку показывает, как механика обслуживания посетителей может быть связана с повествованием и основным геймплеем. В игре приготовление напитков не является отдельным элементом, а помогает раскрывать характеры персонажей и поддерживать атмосферу места, где происходит действие.

- Сочетание игрового процесса и повествования;
- Визуальный стиль и характерные реплики персонажей;
- Влияние действий игрока на развитие диалогов.

Недостатки:

8

– Ограниченность игровых механик.



Рисунок 1 – Пример игрового процесса «VA-11 Hall-A»

1.2.2 Анализ игры «Coffee Talk»

Игра «Coffee Talk» представляет собой визуальную новеллу, в которой игрок выступает в роли бариста в кофейне. Основной игровой процесс включает приготовление напитков и общение с посетителями, каждый из которых имеет свою историю, настроение и внутренний конфликт.

Данный пример важен для анализа, поскольку в нем кофейня используется не только как место действия, но и как пространство для раскрытия персонажей через диалоги. Для проекта «Сурра» важен именно этот подход: спокойная внешняя обстановка кофейни сочетается с более напряженным или мрачным повествованием.

Преимущества:

- Визуальный стиль и характерные реплики персонажей;
- Понятная механика приготовления напитков;
- Акцент на историях посетителей и взаимодействии с ними.

Недостатки:

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						9	

- Ограниченность игровых механик;
- Ограниченное число вариантов исхода.



Рисунок 2 – Пример игрового процесса «Coffee Talk»

1.2.3 Анализ игры «Yunyun Syndrome!? Rhythm Psychosis»

Игра «Yunyun Syndrome!? Rhythm Psychosis» представляет собой ритм-игру с ярким визуальным стилем и необычной подачей. Игровой процесс строится на своевременном выполнении действий под музыку.

Проект выбран как аналог, поскольку позволяет проанализировать ритм-механику и связь музыки с визуальным восприятием. В рамках проекта «Супра» ритм-сцены рассматриваются не как отдельное испытание, а как часть общего настроения и состояния персонажей.

Преимущества:

- Выразительный визуальный стиль;
- Динамичный и вовлекающий игровой процесс;
- Синхронизация визуала и музыки.

Недостатки:

Инов. № подл.	Подп. и дата	Взам. инв. №	Инов. № дубл.	Подп. и дата	1.2.3 Анализ игры «Yunyun Syndrome!? Rhythm Psychosis»					
					Игра «Yunyun Syndrome!? Rhythm Psychosis» представляет собой ритм-игру с ярким визуальным стилем и необычной подачей. Игровой процесс строится на своевременном выполнении действий под музыку. Проект выбран как аналог, поскольку позволяет проанализировать ритм-механику и связь музыки с визуальным восприятием. В рамках проекта «Супра» ритм-сцены рассматриваются не как отдельное испытание, а как часть общего настроения и состояния персонажей. Преимущества: — Выразительный визуальный стиль; — Динамичный и вовлекающий игровой процесс; — Синхронизация визуала и музыки. Недостатки:					
Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ					Лист
										10

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата



Игра «Glisynth» представляет собой экспериментальную ритм-игру, в которой основной акцент сделан на взаимодействии игрока с аудиовизуальной средой. Игровой процесс строится на синхронизации действий с музыкой и визуальными эффектами.

В рамках сравнения этот проект показывает, как звук, визуал и действия игрока могут работать как единая система. Для «Сирра» это важно, потому что ритм-механика приготовления кофе должна быть связана не только с набором очков, но и с ощущением процесса, атмосферой кофейни, историей и итоговым результатом сцены.

					ТПЖА.090301.957 ПЗ	Лист
Изм	Лист	№ докум.	Подп.	Дата		11

- Акцент на синхронизации музыки и визуала;
- Звук синхронизирован с визуальными эффектами;
- Атмосферность и экспериментальность.

Недостатки:

- Отсутствие сюжета;
- Ограниченность игровых целей и мотивации.



Рисунок 4 – Пример игрового процесса «Glisynth»

1.3 Требования к разработке

Требования к разрабатываемому программному продукту можно условно разделить на функциональные и нефункциональные. Первые описывают основные возможности системы и игровой процесс, тогда как вторые определяют качество работы, визуальную составляющую и технические ограничения проекта.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата										
					Рисунок 4 – Пример игрового процесса «Glisynth»									
					1.3 Требования к разработке									
					Требования к разрабатываемому программному продукту можно условно разделить на функциональные и нефункциональные. Первые описывают основные возможности системы и игровой процесс, тогда как вторые определяют качество работы, визуальную составляющую и технические ограничения проекта.									
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ									
										Лист				
Изм	Лист	№ докум.	Подп.	Дата										

1.3.1 Функциональные требования

- а) Система диалогов и повествования: отображение текстовой информации, реплик персонажей и сюжетных сцен, необходимых для развития истории;
- б) Ритм-механика приготовления напитков: реализация игрового процесса, основанного на попадании в ритм музыкального сопровождения;
- в) Синхронизация аудио и визуала: минимальная задержка между звуком и отображением элементов;
- г) Обработка игровых событий: управление ключевыми событиями игрового цикла;
- д) Система оценки и прогрессии: определение точности действий игрока с последующим начислением очков, влияющих на качество приготовления напитков и концовку игры;
- е) Связь игровых элементов: реализация взаимодействия с посетителями по схеме «заказ – приготовление – отдача», связывающей элементы визуальной новеллы и ритм-игры;
- ж) Система состояний: управление переходами между различными режимами игры (меню, диалоги, ритм-игра) без потери текущего прогресса;
- з) Пользовательский интерфейс: отображение диалогов, элементов ритм-игры, количества очков и текущего состояния игрового процесса.

1.3.2 Нефункциональные требования

- а) Визуальная стилистика: 2D-графика в стиле пиксель-арт с вечерней вампирской атмосферой, мягким свечением и контрастом;
- б) Музыкальное сопровождение: музыка с четко выраженным ритмом, подходящая для ритм-сцен;
- в) Производительность: средний FPS не ниже X, 1% low не ниже Y на конфигурации Z;
- г) Кроссплатформенность: корректная и эквивалентная работа игровых механик, интерфейса и переходов на Linux и Windows по тестам;

Изм.	Лист	№ докум.	Подп.	Дата	ж) Система состояний: управление переходами между различными режимами игры (меню, диалоги, ритм-игра) без потери текущего прогресса; з) Пользовательский интерфейс: отображение диалогов, элементов ритм-игры, количества очков и текущего состояния игрового процесса.
1.3.2 Нефункциональные требования					а) Визуальная стилистика: 2D-графика в стиле пиксель-арт с вечерней вампирской атмосферой, мягким свечением и контрастом; б) Музыкальное сопровождение: музыка с четко выраженным ритмом, подходящая для ритм-сцен; в) Производительность: средний FPS не ниже X, 1% low не ниже Y на конфигурации Z; г) Кроссплатформенность: корректная и эквивалентная работа игровых механик, интерфейса и переходов на Linux и Windows по тестам;
ТПЖА.090301.957 ПЗ					
					Лист
					13

- д) Надежность: устойчивость системы к действиям пользователя, способным привести к сбоям;
- е) Читаемость текста: использование элегантных шрифтов, подходящих стилю вампиров, при этом сохраняя удобство восприятия;
- ж) Удобство взаимодействия: интуитивно понятная структура интерфейса и однозначная визуальная обратная связь на действия игрока.

1.4 Постановка задач

В рамках данного курсового проекта ставится задача по разработке функционального прототипа однопользовательской 2D-игры «Супра». Игра будет выполнена в формате визуальной новеллы с элементами ритм-игры, в которой пользователь будет выступать в роли бариста вампирской кофейни, взаимодействовать с посетителями, проходить сюжетные сцены и готовить напитки через ритм-механику.

Ключевая особенность проекта заключается в сочетании двух жанров и внешне спокойной атмосферы кофейни с напряженным повествованием. Качество прохождения ритм-сцен влияет на набор очков, состояние истории и итоговую концовку. Конечной целью является создание интерактивного продукта, демонстрирующего работу системы отображения диалогов, ритм-механики, системы оценки действий игрока и переходов между основными игровыми состояниями.

Разрабатываемый прототип должен включать:

- а) Главное меню с возможностью запуска игры;
- б) Диалоговые сцены визуальной новеллы с отображением персонажей, текста и сюжетных событий;
- в) Ритм-сцену приготовления напитка с обработкой нот;
- г) Систему оценки точности действий игрока и начисления очков;
- д) Систему переходов между меню, сюжетными сценами, ритм-игрой и экраном концовки;
- е) Несколько вариантов концовок, зависящих от результатов прохождения;
- ж) Пользовательский интерфейс, отображающий необходимые игровые данные и обеспечивающий понятное взаимодействие с игрой.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						14	

1.5 Заключение

В данном разделе пояснительной записки была раскрыта предметная область разработки однопользовательской 2D-игры «Сирра». Были рассмотрены особенности ритм-игр и визуальных новелл, а также причины выбора их сочетания в рамках одного проекта.

В ходе анализа были изучены существующие игры-аналоги, выделены их преимущества и недостатки, которые могут быть учтены при разработке собственного программного продукта. На основе проведенного обзора были сформулированы функциональные и нефункциональные требования, определяющие основные возможности игры, ее технические ограничения и ожидаемый пользовательский опыт.

Таким образом, по результатам анализа была сформирована постановка задачи на разработку функционального прототипа игры «Сирра», включающего сюжетные сцены, ритм-механику приготовления напитков, систему оценки действий игрока и переходы между ключевыми состояниями игры.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										15
Изм.	Лист	№ докум.	Подп.	Дата						

2 Проектирование

В данном разделе будет разработана структура однопользовательской 2D-игры «Сирра», а также будут описаны основные состояния, игровые данные и алгоритмы, обеспечивающие работу визуальной новеллы и ритм-механики. Проектирование необходимо для того, чтобы заранее определить связь между меню, сюжетными сценами, ритм-игрой, системой оценки и концовками.

Для данного проекта особенно важно сохранить целостность игрового процесса. Ритм-сцена не должна существовать отдельно от истории, так как именно через приготовление напитков пользователь получает результат, влияющий на дальнейшее повествование. Поэтому структура игры была разделена на несколько связанных модулей.

При описании архитектуры используются следующие уровни ответственности:

- а) Модуль – самостоятельная функциональная часть системы со своей сценарной или координирующей логикой;
- б) Подмодуль – вспомогательная часть, реализующая отдельную подсистему внутри модуля;
- в) Общий компонент – переиспользуемый элемент, который подключается из нескольких модулей и не относится только к одному из них.

С учетом данного разделения структура игры включает:

- а) Модуль главного меню – menu.gd: запуск игры, открытие настроек и выход из приложения;
- б) Подмодуль настроек – settings_overlay.gd: изменение громкости музыкального сопровождения через отдельное окно. Используется в модуле главного меню;
- в) Подмодуль паузы – pause_overlay.gd: отображение внутриигрового меню паузы по клавише Esc (продолжить, настройки, переход в меню, выход). Используется в модуле основной новеллы и координирующем модуле ритм-игры;
- г) Модуль пролога – prologue.gd: начальное введение пользователя в мир игры и подготовка перехода к основной сцене новеллы;
- д) Модуль основной новеллы – novel.gd: отображение персонажей, реплик и переходов между сюжетными фазами;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						16	

е) Модуль концовок – ending.gd: выбор и отображение финала на основе результатов прохождения;

ж) Координирующий модуль ритм-игры – rhythm_game.gd: управление музыкальной сценой, запуском нот, обработкой ввода и завершением ритм-сцены;

з) Подмодуль обработки файлов нот – sm_parser.gd: чтение .sm-файлов и преобразование данных в подходящий формат. Используется координирующим модулем ритм-игры;

и) Подмодуль нот и их поведения – note.gd, hold_note.gd: создание обычных и длинных нот, их движение и обработка удержания;

к) Подмодуль оценки действий игрока – judgment_lane.gd, score_service.gd: определение точности попаданий, начисление очков и изменение комбо;

л) Подмодуль управления подсветкой ламп – side_lights_controller.gd: расчет и обновление визуальных параметров бокового освещения в ритм-сцене;

м) Подмодуль библиотеки композиций – songs_library.gd: хранение набора треков и их параметров для ритм-игры;

н) Общий компонент глобального состояния – global_state.gd: хранение общих игровых данных (очки, комбо, текущий клиент, флаги прогресса, параметры продолжения без сохранения в файл), используемых в разных модулях;

о) Общий UI-компонент фоновой анимации – floating_shapes_layer.gd: визуальный фоновый эффект летающих светлых частиц, переиспользуемый в сценах ритм-игры, меню и новеллы;

п) Общий UI-компонент подсказок управления – controls_hint.gd: оверлей с инструкциями, подключаемый в ритм-сцене.

На рисунке 5 представлена диаграмма взаимосвязи модулей.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						17	

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

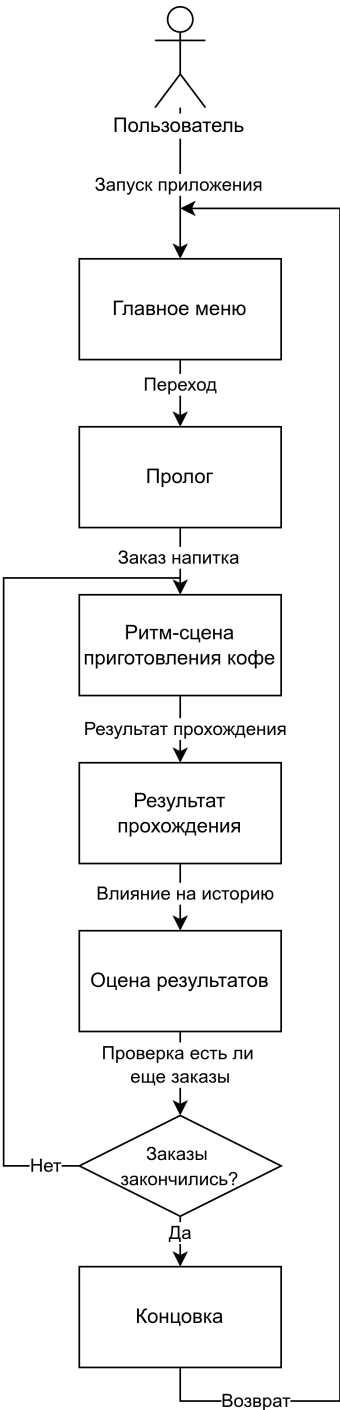


Рисунок 6 – Структура игрового процесса игры «Сирра»

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Начальным состоянием является главное меню. Из него игрок может начать игру, открыть настройки или выйти из приложения. При запуске игры открывается пролог, после чего начинается основная сцена новеллы. После завершения вступительной части для текущего посетителя выполняется переход в ритм-игру.

На рисунке 7 представлена диаграмма состояний игры.

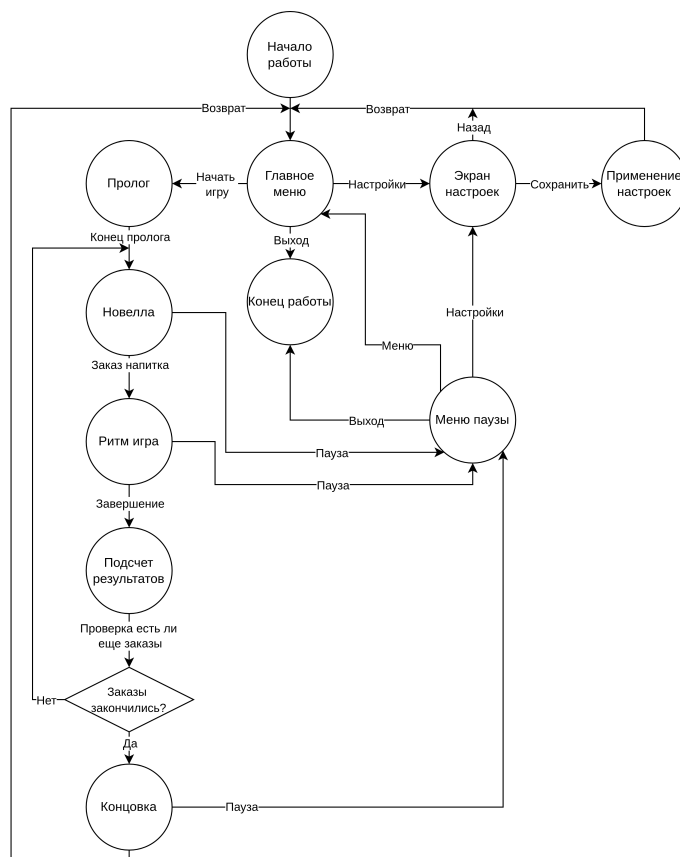


Рисунок 7 – Диаграмма состояний игры «Сурра»

2.3 Проектирование основных алгоритмов

При проектировании алгоритмов и разборе практических приемов реализации дополнительно использовались обучающие видеоматериалы YouTube [20][21][22][23][24][25][26].

2.3.1 Проектирование главного меню и настроек

Главное меню проектировалось как первая точка взаимодействия пользователя с игрой. В нем предусмотрены три основных действия:

- а) Начать игру;
- б) Открыть настройки;
- в) Выйти из приложения.

При наведении на пункты меню меняется цвет и масштаб текста, что обеспечивает визуальную обратную связь и показывает, что с элементом можно взаимодействовать. При выборе начала игры открывается сцена пролога, а при выборе выхода приложение завершает работу.

Настройки реализованы в виде отдельного окна, которое открывается поверх текущей сцены меню. В настройках пользователь может изменить громкость музыки с помощью слайдера. При изменении слайдера сначала меняется временное значение, а итоговое значение сохраняется только после нажатия кнопки сохранения. Если пользователь закрывает окно или нажимает Esc, изменения не применяются.

Для текущего прототипа достаточно регулировки громкости, так как для ритм-игры ключевое значение имеет музыкальное сопровождение.

Инв. № подл.	Подп. и дата				ТПЖА.090301.957 ПЗ	Лист
	Инв. № дубл.					21
	Взам. инв. №					
	Подп. и дата					
Изм.	Лист	№ докум.	Подп.	Дата		

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- а) Вступительная сцена посетителя;
- б) Сцена результата после приготовления напитка.

В новелле не используется выбор реплик пользователем. Основная задача этой части – отображать историю, персонажей и последствия прохождения ритм-сцен. Продвижение по репликам выполняется нажатием клавиши Space или левой кнопки мыши. Также предусмотрена кнопка пропуска, которая позволяет быстрее перейти к следующей фазе.

Для персонажей используются отдельные состояния до и после приготовления напитка. В сцене результата отображается персонаж с кофе. Это помогает визуально показать, что ритм-сцена связана с происходящим в истории, а не является отдельным элементом.

Ритм-уровни проектировались вручную с помощью приложения ArrowVortex [14], после чего сохранялись в формате .sm. Для каждой композиции формируется массив нотных событий, где для каждой ноты задаются моменты появления, номер дорожки и тип ноты.

- а) Аудиофайл – музыкальная композиция, сопровождающая ритм-сцену;
- б) Путь к файлу нот – расположение .sm файла внутри проекта, например
res://RhythmGame/Data/Songs/CrimsonPulse.sm;

					ТПЖА.090301.957 ПЗ	Лист
Изм	Лист	№ докум.	Подп.	Дата		22

в) BPM – количество ударов в минуту, по которому рассчитывается длительность одного бита;

г) Offset – временное смещение карты нот относительно аудио, необходимое для синхронизации музыки и появления нот;

д) Количество нот – значение, используемое для вычисления точности прохождения композиции.

Выбор песни зависит от текущего клиента. Такой подход позволяет связать музыкальное сопровождение с конкретной сценой и настроением персонажа.

В ритм-игре используются четыре дорожки, соответствующие клавишам S, D, K и L. Ноты двигаются к линии оценки, а пользователь должен нажимать нужные клавиши в подходящий момент. Ниже введены обозначения и единицы измерения:

- BPM – темп композиции, уд/мин;
- t_{beat} – длительность одного бита, с;
- $offset$ – смещение нотной карты относительно начала аудио, с;
- y_{spawn} – координата точки появления ноты, пиксели;
- y_{judge} – координата линии оценки, пиксели;
- N_{travel} – число битов, за которое нота должна дойти до линии оценки;
- t_{move} – время движения ноты от точки появления до линии оценки, с;
- d – расстояние от точки появления до линии оценки, пиксели;
- v – скорость движения ноты, пикс/с;
- b_i – позиция i -й ноты в битах от начала карты (например, $b_i = 0$ – начало карты, $b_i = 1$ – через один бит, $b_i = 2.5$ – через два с половиной бита);
- t_i – момент оценки i -й ноты на линии, с;
- t_i^{spawn} – момент создания i -й ноты на сцене, с.

Чтобы ноты появлялись заранее и успевали дойти до линии оценки, выполняются следующие расчеты:

а) Расчет длительности одного бита. Длительность одного бита определяется через темп композиции по формуле (2.1):

$$t_{beat} = \frac{60}{BPM}. \quad (2.1)$$

б) Расчет времени движения ноты по формуле (2.2).

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата					
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ				
					Лист				
					23				

$$t_{move} = N_{travel} \cdot t_{beat}, \quad (2.2)$$

где N_{travel} – константа числа битов до линии оценки.

в) Расчет расстояния от точки появления до линии оценки. Расстояние берется по вертикальной оси по модулю, потому что длина пути всегда должна быть положительной, по формуле (2.3):

$$d = |y_{spawn} - y_{judge}|. \quad (2.3)$$

г) Расчет скорости движения ноты. Скорость выбирается такой, чтобы нота проходила расстояние d за время t_{move} , по формуле (2.4):

$$v = \frac{d}{t_{move}}. \quad (2.4)$$

д) Расчет времени оценки и времени появления каждой ноты. Для i -й ноты сначала определяется момент, когда она должна оказаться на линии оценки, по формуле (2.5):

$$t_i = offset + b_i \cdot t_{beat}. \quad (2.5)$$

Здесь $offset$ задает общее смещение всей карты относительно начала аудио, а $b_i \cdot t_{beat}$ переводит позицию ноты из битов в секунды.

е) Момент создания ноты на сцене. То есть нота появляется раньше момента оценки ровно на время своего движения, по формуле (2.6):

$$t_i^{spawn} = t_i - t_{move}. \quad (2.6)$$

Эти параметры обеспечивают синхронизацию музыки и движения нот.

Перед первым запуском ритм-сцены отображается подсказка управления. Она появляется только один раз, чтобы пользователь понял, какие клавиши используются в игре. После подтверждения подсказки запускается ритм-сцена, а подсказка закрывается.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ Изм. Лист № докум. Подп. Дата Лист 24					

2.5 Проектирование алгоритма обработки файла нот

Для генерации нот используется файл формата .sm. Он содержит данные о расположении обычных и длинных нот. Перед началом ритм-сцены файл необходимо прочитать и преобразовать в массив игровых событий, с которым может работать модуль ритм-сцены без дополнительной подготовки.

На рисунке 8 представлена последовательность действий обработки .sm файла.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата					
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ			Лист	
								25	

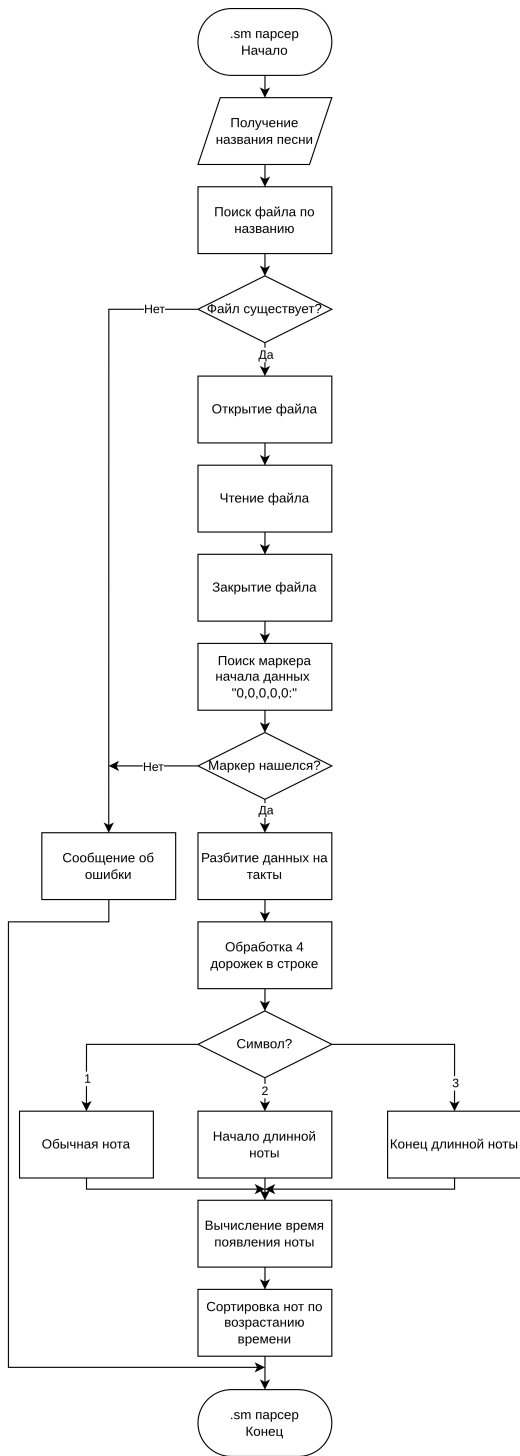


Рисунок 8 – Последовательность работы подмодуля обработки файлов нот

Данный алгоритм включает несколько последовательных проверок и циклов. Сначала выполняются две обязательные проверки: удалось ли открыть файл и найден ли маркер начала нотных данных 0,0,0,0,0:. Если хотя бы одно из этих условий не выполнено, подмодуль завершает работу досрочно и возвращает пустой массив событий.

После успешной проверки запускается внешний цикл по тактам композиции. Внутри каждого такта выполняется цикл по строкам, из которых отбираются только строки длиной 4 символа, соответствующие четырем дорожкам. Если после фильтрации строк в такте не осталось строк, выполняется переход к следующему такту. Далее для валидного такта запускается цикл по строкам и вложенный цикл по дорожкам от 1 до 4, где каждый символ интерпретируется как тип события.

В этой части используются ветвления по символу ноты: при 1 сразу создается событие обычной ноты, при 2 запоминается время начала длинной ноты, при 3 сначала проверяется наличие сохраненного начала удержания для данной дорожки, после чего создается событие длинной ноты с рассчитанной длительностью. После обработки каждой строки увеличивается внутреннее игровое время, что обеспечивает синхронизацию будущих нот с ритмом.

2.6 Проектирование алгоритма ритм-игры

Основная ритм-сцена отвечает за запуск музыки, генерацию нот, обработку игрового цикла и завершение раунда. Перед началом сцены определяется текущий клиент, по нему выбирается песня, после чего рассчитываются параметры движения нот. К таким параметрам относятся длительность одного бита, время движения ноты к линии оценки и скорость движения.

На рисунке 9 представлена схема алгоритма ритм-игры.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

игрового цикла и завершение раунда. Перед началом сцены определяется текущий клиент, по нему выбирается песня, после чего рассчитываются параметры движения нот. К таким параметрам относятся длительность одного бита, время движения ноты к линии оценки и скорость движения.

На рисунке 9 представлена схема алгоритма ритм-игры.

Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ	Лист
						27

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата
Изм.	Лист	№ докум.	Подп.	Дата

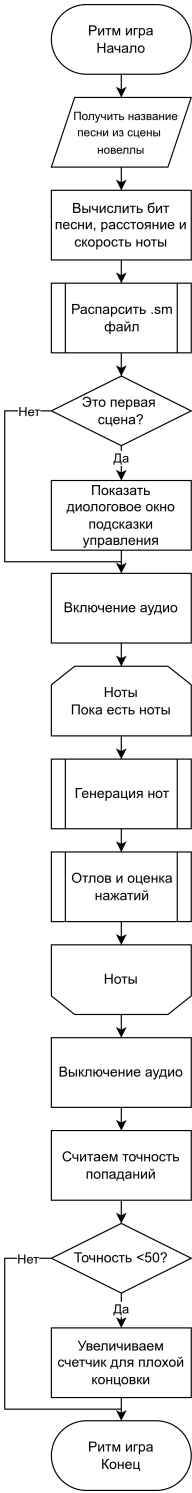


Рисунок 9 – Схема алгоритма ритм-игры

После подготовки данных сцена проверяет, показывалась ли игроку подсказка управления. Если подсказка еще не была показана, игра переходит в режим ожидания подтверждения, в котором основной ритм-цикл временно не выполняется. После подтверждения подсказки запускается основной цикл ритм-игры. В нем обновляется время, проверяется необходимость создания новых нот, запускается

аудио и обновляется интерфейс. Дополнительно во время ритм-сцены поддерживается пауза по клавише Esc с возможностью продолжить игру, открыть настройки, перейти в меню или выйти из приложения.

Ноты создаются не в момент их оценки, а заранее. Это нужно для того, чтобы они успели пройти путь от точки появления до линии оценки. Если в очереди есть нота, время которой уже наступило, система создает обычную или длинную ноту на нужной дорожке. Завершение раунда выполняется только при одновременном выполнении нескольких условий: музыка уже была запущена, воспроизведение завершено, очередь нот пуста, на сцене нет активных нот, а флаг завершения еще не установлен. Такая проверка снижает риск преждевременного или повторного завершения одной и той же ритм-сцены.

По окончании ритм-сцены рассчитывается точность прохождения. Для этого набранные очки сравниваются с максимально возможным количеством очков за обработанные ноты. Если точность ниже 50%, результат считается слабым и увеличивает счетчик плохих результатов. Затем игра возвращается в сцену новеллы и показывает сюжетную реакцию персонажа.

2.7 Проектирование обычных и длинных нот

В ритм-игре используются два типа нот: обычные и длинные. Обычная нота требует одного своевременного нажатия. После попадания она оценивается, удаляется со сцены, а игрок получает очки в зависимости от точности.

Длинная нота устроена иначе. Для нее важен не только момент начала, но и удержание клавиши. При попадании по началу длинной ноты она переходит в состояние удержания. Пока пользователь удерживает клавишу, тело ноты постепенно уменьшается. Если игрок удерживал ноту хоть какое-то время, он получает оценку за длительность удержания. Если нота не была начата, результат переходит в состояние пропуска.

Такой тип нот расширяет набор действий игрока за счет механики удержания. Обычные ноты проверяют своевременное нажатие, а длинные требуют удержания клавиши в течение заданного времени. Это помогает связать действия игрока с ритмом музыкального сопровождения и уменьшить однотипность нажатий.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	2.7 Проектирование обычных и длинных нот					
					В ритм-игре используются два типа нот: обычные и длинные. Обычная нота требует одного своевременного нажатия. После попадания она оценивается, удаляется со сцены, а игрок получает очки в зависимости от точности. Длинная нота устроена иначе. Для нее важен не только момент начала, но и удержание клавиши. При попадании по началу длинной ноты она переходит в состояние удержания. Пока пользователь удерживает клавишу, тело ноты постепенно уменьшается. Если игрок удерживал ноту хоть какое-то время, он получает оценку за длительность удержания. Если нота не была начата, результат переходит в состояние пропуска. Такой тип нот расширяет набор действий игрока за счет механики удержания. Обычные ноты проверяют своевременное нажатие, а длинные требуют удержания клавиши в течение заданного времени. Это помогает связать действия игрока с ритмом музыкального сопровождения и уменьшить однотипность нажатий.					
					ТПЖА.090301.957 ПЗ					Лист
										29
Изм	Лист	№ докум.	Подп.	Дата						

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

На рисунке 10 представлена схема алгоритма оценки действий игрока.

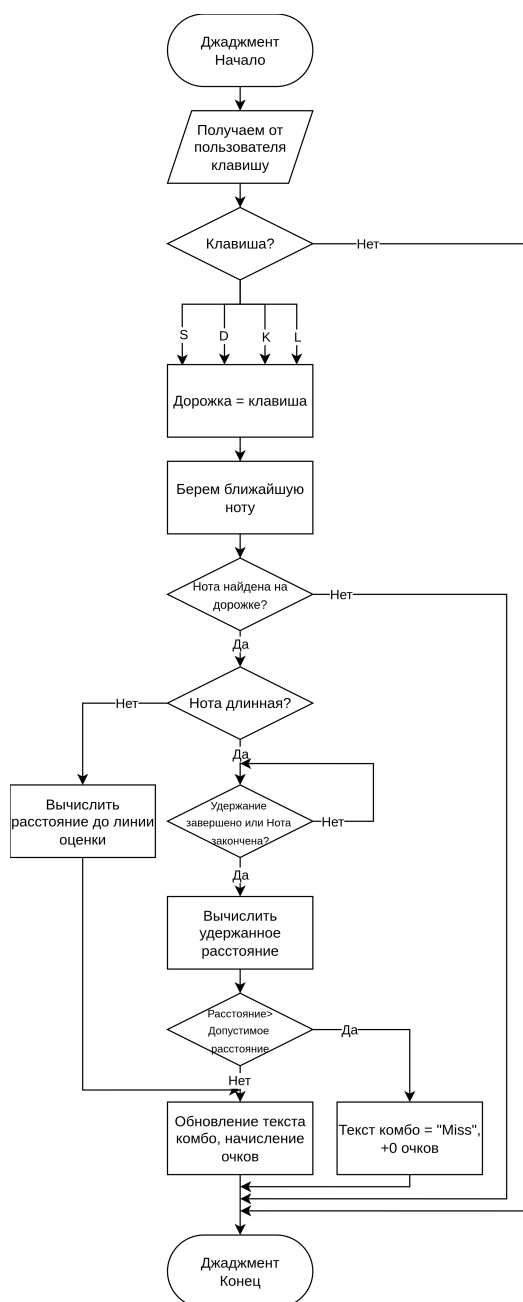


Рисунок 10 – Схема алгоритма оценки действий игрока.

После нажатия клавиши система определяет дорожку и ищет ближайшую ноту в зоне оценки. Если нота не найдена, нажатие не обрабатывается. Если нота найдена, рассчитывается расстояние между нотой и линией оценки, после чего выставляется результат попадания. Дополнительно на дорожке отображается цветовая обратная связь, чтобы пользователь сразу видел итог своего действия. За счет этого ритм-сцена становится понятнее: игрок получает не только числовую оценку, но и визуальный отклик.

В проекте используются следующие оценки попадания:

- а) Bloody bite – лучшая оценка, визуально подсвечивается ярко-розовым цветом;
- б) Bloody – хорошая оценка, визуально подсвечивается ярко-оранжевым цветом;
- в) Lame – базовая оценка, визуально подсвечивается ярко-бирюзовым цветом;
- г) Miss – нота пропущена, худшая оценка, визуально подсвечивается темно-фиолетовым цветом.

Критерии оценок и баллов для обычных нот:

- а) Bloody bite – расстояние меньше 50 пикселей, начисляется 10 очков;
- б) Bloody – расстояние меньше 80 пикселей, начисляется 5 очков;
- в) Lame – расстояние меньше 110 пикселей, начисляется 1 очко;
- г) Miss – нота пропущена, очки не начисляются, комбо сбрасывается.

При успешном попадании значение комбо увеличивается на 10. Если игрок пропускает ноту, комбо сбрасывается.

Для длинных нот оценка выставляется по прогрессу удержания (отношению удержанной части к полной длине ноты):

- а) Bloody bite – прогресс удержания не менее 0.85, начисляется 10 очков;
- б) Bloody – прогресс удержания не менее 0.50, начисляется 5 очков;
- в) Lame – прогресс удержания меньше 0.50, начисляется 1 очко;
- г) Miss – длинная нота не была начата вовремя и вышла из зоны оценки, очки не начисляются.

Такой подход позволяет учитывать не только точность начального нажатия, но и длительность удержания в ритме музыкального сопровождения.

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

2.9 Проектирование алгоритма выбора концовки

Концовки проектировались как итог связи ритм-механики и повествования. Игрок не выбирает финал напрямую. Вместо этого итог зависит от качества прохождения ритм-сцен.

После каждой ритм-сцены система рассчитывает точность попаданий в процентах. Сначала определяется максимально возможный балл для уже обработанных нот: количество обработанных нот умножается на 10 (максимум за одну ноту). Затем фактически набранные очки делятся на этот максимум и переводятся в проценты по формуле (2.7):

$$Accuracy = \frac{Score}{JudgedCount \cdot 10} \cdot 100\%. \tag{2.7}$$

Если обработанных нот нет, точность принимается равной 0%. Если полученная точность меньше 50%, текущий результат считается слабым, и счетчик плохих результатов увеличивается на единицу.

На рисунке 11 представлена схема алгоритма выбора концовки.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
--------------	--------------	--------------	--------------	--------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

2.10 Заключение

В данном разделе была рассмотрена проектная часть однопользовательской 2D-игры «Сирра». Были описаны структура игрового процесса, диаграмма состояний, структура главного меню и настроек, организация сюжетных сцен и ритм-уровней, а также алгоритмы обработки нот, оценки действий игрока и выбора концовки.

В ходе проектирования было определено, что основой игры является переход между сюжетными сценами и ритм-сценами приготовления кофе. Ритм-механика обеспечивает взаимодействие пользователя с музыкой, а сюжетная часть связывает результаты прохождения с персонажами и финалом истории.

Таким образом, проектирование позволило сформировать целостную структуру игры, в которой меню, настройки, новелла, ритм-игра, система очков и концовки работают как связанные части одного игрового процесса.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ					Лист
										34

- res://Menu/Scenes/ – сцены меню, настроек и паузы;
- res://Menu/Scripts/Menu/ – логика главного меню;
- res://Menu/Scripts/Settings/ – логика окна настроек;
- res://Menu/Scripts/Pause/ – логика паузы;
- res://Menu/Assets/Background/ – фоновые изображения меню;
- res://Menu/Data/Songs/ – музыкальные ресурсы.
- res://Novel/ – каталог реализации модулей пролога, основной новеллы и концовок.

- res://Novel/Scenes/ – сцены пролога, основной новеллы и концовки;
- res://Novel/Scripts/Flow/ – логика фаз истории;
- res://Novel/Scripts/UI/ – логика подсказки в начале ритм-игры;
- res://Novel/Assets/Backstory/ – фоновые изображения пролога;
- res://Novel/Assets/Novel/ – фоновые изображения новеллы;
- res://Novel/Assets/Sprites/ – спрайты персонажей;
- res://Novel/Data/Dialog/ – данные диалогов;
- res://Novel/Data/Songs/ – музыкальные ресурсы.
- res://RhythmGame/ – каталог реализации координирующего модуля ритм-игры и его подмодулей.

- res://RhythmGame/Scenes/ – игровая сцена, сцены нот и линии оценки;
- res://RhythmGame/Scripts/Controllers/ – управляющая логика ритм-раунда;

- res://RhythmGame/Scripts/Gameplay/ – основная игровая логика;
- res://RhythmGame/Scripts/Parsing/ – логика парсера .sm файлов;
- res://RhythmGame/Scripts/State/ – хранение игровых данных;
- res://RhythmGame/Scripts/UI/ – логика визуальных эффектов ритм-сцены;
- res://RhythmGame/Assets/Sprites/ – изображения фона, нот и линии оценки;
- res://RhythmGame/Assets/VFX/Effects/ – визуальные эффекты;
- res://RhythmGame/Data/Songs/ – музыкальные ресурсы.

В данной структуре проекта каждый модуль и подмодуль реализует четко определенную функцию, а каталоги проекта отражают эту границу ответствен-

ТПЖА.090301.957 ПЗ

ности на уровне кода и ресурсов. Это повышает масштабируемость, упрощает навигацию и снижает сложность сопровождения.

3.3 Реализация игрового движка

В рамках реализации игрового движка была собрана целостная игровая система, где визуальные и звуковые ресурсы, интерфейс и модули игровой логики работают как связанные части единого процесса.

3.3.1 Создание игровых ресурсов

Для подготовки контента был выделен отдельный этап, так как именно визуальная и звуковая часть определяют восприятие игрового мира и обеспечивают читаемую обратную связь.

Создание графических ресурсов включало:

- а) Ручную отрисовку базовых ассетов в графическом редакторе Aseprite [15];
- б) Подготовку промптов в ChatGPT [16] и генерацию анимаций по отрисованному материалу, а также генерацию отдельных изображений в Leonardo.AI [17] с использованием разных моделей. Сгенерированные изображения переводились в пиксельный формат с помощью отдельного конвертера в режиме Mono, после чего дорабатывались вручную в Aseprite.

Создание звуковых ресурсов включало:

- а) Подготовку промптов в ChatGPT [16] и генерацию музыкального сопровождения в Suno [18] с последующей ручной обработкой в аудиоредакторе Audacity [19].
- Созданные ресурсы включают:
- а) Спрайты персонажей и их состояний;
 - б) Локации и элементы окружения;
 - в) Обычные и длинные ноты;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						37	

фона нужно для сохранения контекста текущей сцены и визуального фокусирования на настройках; Panel задает границы рабочей области, а контейнеры VBoxContainer/HBoxContainer обеспечивают аккуратную компоновку элементов.

Три Label отвечают за заголовок «Настройки», подпись параметра «Музыка» и отображение значения громкости в процентах; HSlider изменяет громкость, два Button отвечают либо за подтверждение и применение параметра слайдера, либо за выход без подтверждения с помощью кнопки «Назад». Ползунок изменяет черновое значение громкости, а фактическое применение изменений выполняется только по команде «Сохранить», что снижает риск случайных изменений и делает поведение окна предсказуемым.

Внешний вид окна настроек показан на рисунке 12.



Рисунок 12 – Окно настроек

3.3.4 Реализация паузы

Подмодуль паузы реализован в сцене Menu/Scenes/PauseOverlay.tscn как отдельный оверлей на CanvasLayer со скриптом Menu/Scripts/Pause/pause_overlay.gd. Использование ColorRect с размытием и затемняющим слоем нужно для того, чтобы остановить игровой контекст визуально, но сохранить его читаемость; Panel и контейнер VBoxContainer формируют компактное размещение элементов формы

в центре экрана. При открытии оверлея сцена сзади переводится в режим паузы, что предотвращает фоновое выполнение игровой логики.

В качестве интерактивных элементов используются несколько Button: «Продолжить», «Настройки», «В меню» и «Выйти». Кнопка «Продолжить» возвращает пользователя в активный игровой процесс, «Настройки» открывает SettingsOverlay поверх паузы, команда «В меню» сохраняет путь текущей сцены для последующего продолжения, а «Выйти» завершает работу игры. Тем самым пауза выступает не только как остановка игры, но и как общий интерфейс управления вспомогательными действиями.

Внешний вид окна внутриигрового меню показан на рисунке 13.



Рисунок 13 – Окно внутриигрового меню

3.3.5 Реализация пролога

Пролог реализован в сцене Novel/Scenes/Prologue.tscn на базе Node2D с подключением скрипта Novel/Scripts/Flow/prologue.gd.

Текущая экранная форма пролога вынесена в приложение А, рисунок А.2.

В сцене используются Sprite2D, выступающие в роли фона, а также два Label – Role и Dialoge, отвечающие за отображение роли говорящего и текущей реплики. За музыкальное сопровождение отвечает AudioStreamPlayer2D с автозапуском за- цикленного трека Prologue. Такая структура поддерживает атмосферу вступления и обеспечивает отображение текста.

Интерактивная часть построена на трех каналах управления: клавиша Space, левая кнопка мыши и кнопка «Пропустить». В prologue.gd массив реплик за- грузывается из prologue_lines.gd, после чего функции show_line() и next_line() последовательно обновляют содержимое Role и Dialoge. При достижении конца массива или при нажатии «Пропустить» запускается единый переход к следующей сцене.

Для смены сцены используется плавное затемнение, что убирает резкий визуальный разрыв между эпизодами. Дополнительно при запуске пролога обнов- ляется глобальное состояние продолжения, поэтому пользователь может корректно вернуться к текущему этапу из главного меню.

3.3.6 Реализация новеллы

Основная новелла реализована в сцене Novel/Scenes/MainNovel.tscn на базе Node2D с подключением скрипта Novel/Scripts/Flow/novel.gd.

Текущая экранная форма основной новеллы вынесена в приложение A, рису- нок A.3.

В сцене используются AnimatedSprite2D для фона и персонажей, Panel для области диалога, Label – Role и DialogText для вывода роли и текста реплики, Button – «Пропустить» для пропуска сцены новеллы, AudioStreamPlayer2D с автозапуском зацикленного трека Novel.

В модуле используются все спрайты персонажей и их состояний. Примеры спрайтов показаны на рисунках 15, 16, 17, 18, 19, 20.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Основная новелла реализована в сцене Novel/Scenes/MainNovel.tscn на базе Node2D с подключением скрипта Novel/Scripts/Flow/novel.gd.					
					Текущая экранная форма основной новеллы вынесена в приложение А, рисунок А.3.					
					В сцене используются AnimatedSprite2D для фона и персонажей, Panel для области диалога, Label – Role и DialogText для вывода роли и текста реплики, Button – «Пропустить» для пропуска сцены новеллы, AudioStreamPlayer2D с автозапуском зацикленного трека Novel.					
					В модуле используются все спрайты персонажей и их состояний. Примеры спрайтов показаны на рисунках 15, 16, 17, 18, 19, 20.					
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ					Лист
										41



Рисунок 15 – Спрайт Мэри



Рисунок 16 – Спрайт Мэри с кофе



Рисунок 17 – Спрайт Кори

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата					
Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ				
					Лист				
					42				



Рисунок 18 – Спрайт Кори с кофе

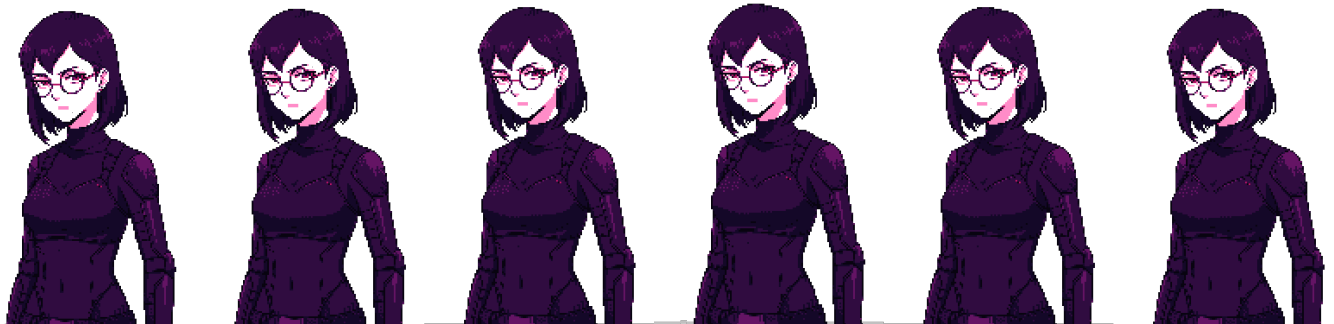


Рисунок 19 – Спрайт Роуз



Рисунок 20 – Спрайт Роуз с кофе

Сценарная логика опирается на данные из Novel/Data/Dialog/dialogs.gd, где диалоги организованы по фазам:

- а) mary_intro;
- б) mary_happy;
- в) mary_sad;
- г) cory_intro;
- д) cory_happy;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата					
Рисунок 20 – Спрайт Роуз с кофе									
Сценарная логика опирается на данные из Novel/Data/Dialog/dialogs.gd, где диалоги организованы по фазам: а) mary_intro; б) mary_happy; в) mary_sad; г) cory_intro; д) cory_happy;									
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ				
					43				

- е) cory_sad;
- ж) rose_intro;
- з) rose_happy;
- и) rose_sad;
- к) rose_bad_ending.

В novel.gd выбирается нужный набор реплик по текущему состоянию новеллы, скрываются неактуальные спрайты и показывается только персонаж текущей фазы.

Интерактивная часть построена на клавише Space, левой кнопке мыши, кнопке «Пропустить» и клавише Esc. Команды Space/ЛКМ переводят диалог на следующую реплику, «Пропустить» пропускает текущую сюжетную сцену и переходит к ритм-игре, а Esc открывает оверлей паузы.

Для смены сцены используется плавное затемнение, что убирает резкий визуальный разрыв между эпизодами. Дополнительно при запуске новеллы обновляется глобальное состояние продолжения, поэтому пользователь может корректно вернуться к текущему этапу из главного меню.

3.3.7 Реализация ритм-игры

Ритм-игра реализована как отдельный модуль, в котором сцены, скрипты парсинга, логика генерации нот, система оценки и визуальные эффекты работают как единый игровой цикл. По структуре проектирования эта часть соответствует координирующему модулю rhythm_game.gd и набору специализированных подмодулей.

Текущая экранная форма ритм-сцены вынесена в приложение А, рисунок А.4.

Сцена раунда реализована в файле RhythmGame/Scenes/RhythmGame.tscn и построена на Node2D.

Сцена включает следующие ключевые элементы:

- а) Точки спавна нот: SpawnPoint1, SpawnPoint2, SpawnPoint3, SpawnPoint4;
- б) Зона оценки: JudgmentRoot;
- в) Аудиоплеер трека: AudioStreamPlayer2D;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										44
Изм	Лист	№ докум.	Подп.	Дата						Формат А4

- г) Индикаторы состояния: Score, Combo, ComboScore, ProgressBar;
 - д) Фон: AnimatedSprite2D;
 - е) Визуальные эффекты: FloatingShapesLayer, LightLeft/LightRight;
 - ж) Оверлей подсказки управления перед первым запуском ритм-сцены.
- Для ритм-объектов используются три отдельные сцены:

а) RhythmGame/Scenes/Note.tscn – обычная нота на базе Area2D с хит-боксом, спрайтом ноты и визуальным эффектом подсвечивания. Внешний вид обычной и длинной ноты показан на рисунке 21. Эффект ноты изображен на рисунке 24;

б) RhythmGame/Scenes/LongNote.tscn – длинная нота, состоящая из головы, тела и хвоста, с отдельным хитбоксом и расчетом длины через длительность удержания. Внешний вид частей длинной ноты показан на рисунке 22;

в) RhythmGame/Scenes/Judgment.tscn – зона оценки и визуальные анимации отклика при попадании. Внешний вид ассета показан на рисунке 23. Эффект судьи изображен на рисунке 25.



Рисунок 21 – Ноты: обычная нота и длинная нота

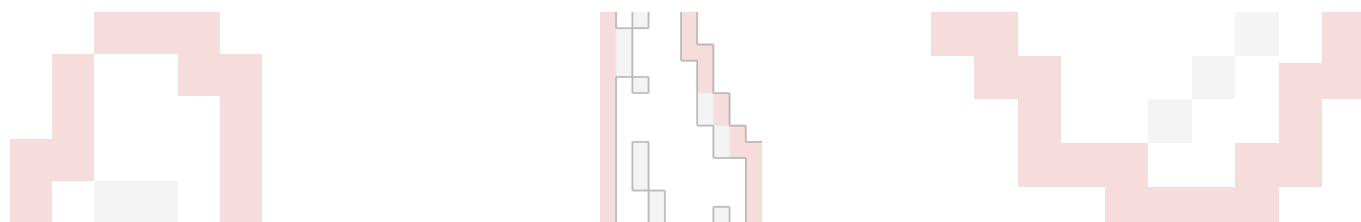


Рисунок 22 – Части длинной ноты: верхняя часть, тело, нижняя часть

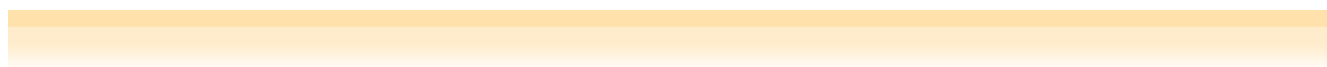
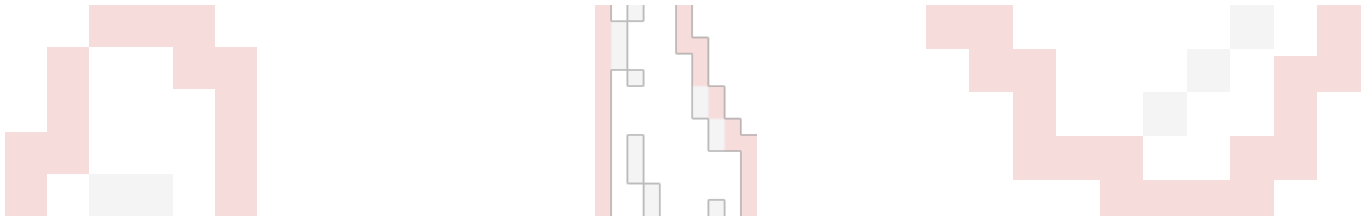
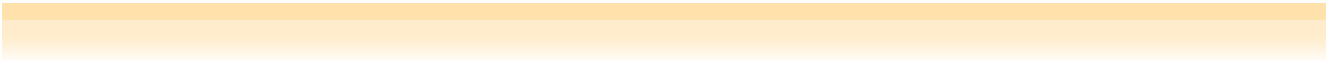


Рисунок 23 – Линия оценки

Инв. № подл.	Подп. и дата				Лист
	Инв. № дубл.				
Взам. инв. №	Подп. и дата				
	Инв. № дубл.				
					
Рисунок 21 – Ноты: обычная нота и длинная нота					
					
Рисунок 22 – Части длинной ноты: верхняя часть, тело, нижняя часть					
					
Рисунок 23 – Линия оценки					

Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ	Лист



Рисунок 24 – Эффект ноты



Рисунок 25 – Эффект нажатия клавиши у дорожки зоны оценки



Рисунок 26 – Боковой свет

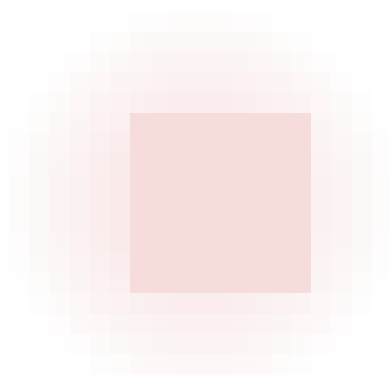


Рисунок 27 – Светящиеся частицы

Перед первым запуском ритм-сцены пользователю показывается отдельная подсказка управления с набором рабочих клавиш. Подсказка отображается только один раз, после подтверждения закрывается и не прерывает последующие раунды.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										46
Изм.	Лист	№ докум.	Подп.	Дата	Копировал					Формат А4

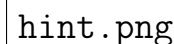


Рисунок 28 – Подсказка управления перед первой ритм-сценой

UI-подмодули визуальной обратной связи.

- а) RhythmGame/Scripts/UI/side_lights_controller.gd – синхронно обновляет прозрачность левых и правых источников света через синусоидальный пульс;
- б) RhythmGame/Scripts/UI/pop_effect.gd – запускает короткую анимацию «пор» для текста комбо и усиливает ощущение отклика;
- в) RhythmGame/Scripts/UI/floating_shapes_layer.gd – формирует слой плавающих частиц со случайным масштабом, скоростью и пульсацией прозрачности, создавая живой фон ритм-сцены.

3.3.8 Реализация концовок новеллы

Модуль концовки реализован в сцене Novel/Scenes/Ending.tscn на базе Node2D с подключением скрипта Novel/Scripts/Flow/ending.gd. Сцена построена так, чтобы в рамках одного экрана поддерживать два варианта финала и единый диалоговый интерфейс.

В состав сцены входят следующие ключевые элементы:

- а) GoodEnding – слой хорошей концовки с фоновым изображением;
- б) BadEnding – полностью затемняющий ColorRect;
- в) Role и DialogText – Label для отображения говорящего и текста реплики;
- г) Button («Дальше») – кнопка выхода из концовки после показа всех реплик.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						47	



Рисунок 32 – Итоговый макет, на основе которого сформирована текущая версия интерфейса

3.6 Экспериментальная апробация

Экспериментальная апробация выполнялась не только как линейное прохождение, но и как серия повторных проверок в разных состояниях интерфейса и игрового цикла.

Проверка базовых пользовательских сценариев:

- а) Запуск игры через главное меню, переход в пролог и переход в основную новеллу;
- б) Прохождение сюжетной сцены с переходом к ритм-раунду;
- в) Обработка обычных и длинных нот, включая старт удержания и досрочное отпускание;
- г) Возврат в новеллу после завершения ритм-сцены;
- д) Проверка формирования игрой двух вариантов концовки.

Проверка интерфейса и устойчивости к повторным действиям:

- а) Многократные нажатия на кнопки меню, настроек и паузы в разных состояниях сцены;
- б) Повторные открытия и закрытия окна настроек из главного меню и из оверлея паузы;
- в) Проверка корректной реакции кнопок на наведение и нажатие;

Подп. и дата		ТПЖА.090301.957 ПЗ Лист 50				
Инв. № дубл.						
Взам. инв. №						
Подп. и дата						
Инв. № подл.		Изм	Лист	№ докум.	Подп.	Дата

г) Серия быстрых кликов по интерактивным элементам для проверки отсутствия некорректных двойных переходов;

д) Проверка одноразового показа подсказки управления перед первой ритм-сценой.

Проверка отказоустойчивости ритм-подсистемы:

а) Загрузка некорректных .sm-файлов и проверка корректной обработки без аварийного завершения;

б) Проверка поведения при отсутствии ожидаемой структуры нотных данных;

Проверка производительности и кроссплатформенного поведения:

а) Проверка стабильности на персональном компьютере без заметных просадок частоты кадров в основных сценах;

б) Проверка запуска и прохождения ключевых сценариев на виртуальной машине с низкими техническими характеристиками без критических задержек ввода и визуальных артефактов;

в) Сравнительная проверка на Linux (среда разработки) и Windows: переходы между сценами, логика управления и обработка ритм-событий дают сопоставимый результат в проверенных сценариях.

По итогам апробации подтверждено прохождение ключевого игрового цикла по сценариям, перечисленным в разделе апробации, отсутствие сбоев интерфейса при серии повторных действий в проверенных сценариях, стабильная обработка некорректных входных данных нотных карт и сопоставимое поведение проекта на Linux и Windows в рамках проверенных сценариев, включая запуск в виртуализированной среде.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист				
										51				
										Изм.	Лист	№ докум.	Подп.	Дата

Заключение

В рамках курсового проекта была разработана однопользовательская 2D-игра «Сирра», объединяющая визуальную новеллу и ритм-механику в едином игровом цикле. Основная цель работы заключалась в создании функционального прототипа, где сюжет, музыкальный геймплей и система оценки действий пользователя связаны между собой и влияют на итог прохождения.

В ходе выполнения работы была проанализирована предметная область, сформулированы функциональные и нефункциональные требования, а также спроектирована модульная структура приложения. Для проекта были выделены модули и подмодули с разделением ответственности: главное меню, настройки, пауза, пролог, основная новелла, ритм-игра, модуль концовок и общие UI-компоненты. Такое разделение было направлено на сохранение целостности архитектуры и упрощение сопровождения кода.

На этапе реализации подготовлены графические и аудиоресурсы, настроены сцены и скрипты управления, реализованы алгоритмы ритм-раунда и обработки обычных и длинных нот; также реализованы система оценки точности и механизм выбора концовки. Отдельное внимание было уделено пользовательскому интерфейсу: добавлены подсказки управления, пауза, настройки и визуальная обратная связь, обеспечивающие предсказуемые пользовательские сценарии.

Экспериментальная апробация подтвердила корректность ключевых сценариев.

Итогом проекта является прототип, прошедший проверку базовых пользовательских сценариев, демонстрирующий реализуемость выбранного подхода к интеграции визуальной новеллы и ритм-игры. Поставленные задачи выполнены в объеме, заявленном во введении, а полученная структура проекта может использоваться как основа для дальнейшего развития: расширения сюжетных веток, улучшение визуала, добавления новых композиций и усложнения ритм-механик без переработки базовой архитектуры.

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм.	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Список литературы

1. Entertainment Software Association. Essential Facts About the U.S. Video Game Industry: 2024. URL: <https://www.theesa.com/wp-content/uploads/2024/05/Essential-Facts-2024-FINAL.pdf> (дата обращения: 27.02.2026).
2. Newzoo. Global Games Market Report 2025 (Free version). URL: <https://newzoo.com/resources/trend-reports/newzoo-global-games-market-report-2025> (дата обращения: 27.02.2026).
3. Steam. Year in Review 2024. URL: <https://steamcommunity.com/groups/steamworks/announcements/detail/751641001553035272> (дата обращения: 27.02.2026).
4. Zhang J., Fu X. The Influence of Background Music of Video Games on Immersion // Journal of Psychology & Psychotherapy. 2015. Vol. 5, Issue 4. URL: <https://www.longdom.org/pdfdownload.php?aid=58693&download=open-access%2Fthe-influence-of-background-music-of-video-games-on-immersion-2161-0487-1000191.pdf> (дата обращения: 27.02.2026).
5. Toh W. The Player Experience and Design Implications of Narrative Games // International Journal of Human-Computer Interaction. 2022. DOI: 10.1080/10447318.2022.2085404. URL: <https://www.tandfonline.com/doi/abs/10.1080/10447318.2022.2085404> (дата обращения: 27.02.2026).
- 6.osu! About osu! Rhythm Game. URL: <https://osu.net/about> (дата обращения: 19.03.2026).
7. StepMania. URL: <https://www.stepmania.com/> (дата обращения: 19.03.2026).
8. Encyclopaedia Britannica. Guitar Hero. URL: <https://www.britannica.com/topic/Guitar-Hero> (дата обращения: 19.03.2026).
9. Steam. VA-11 Hall-A: Cyberpunk Bartender Action. URL: https://store.steampowered.com/app/447530/VA11_Hall_A_Cyberpunk_Bartender_Action/ (дата обращения: 03.04.2026).
10. Steam. Coffee Talk. URL: https://store.steampowered.com/app/914800/Coffee_Talk/ (дата обращения: 03.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата	5. Toh W. The Player Experience and Design Implications of Narrative Games // International Journal of Human-Computer Interaction. 2022. DOI: 10.1080/10447318.2022.2085404. URL: https://www.tandfonline.com/doi/abs/10.1080/10447318.2022.2085404 (дата обращения: 27.02.2026). 6. osu! About osu! Rhythm Game. URL: https://osu.net/about (дата обращения: 19.03.2026). 7. StepMania. URL: https://www.stepmania.com/ (дата обращения: 19.03.2026). 8. Encyclopaedia Britannica. Guitar Hero. URL: https://www.britannica.com/topic/Guitar-Hero (дата обращения: 19.03.2026). 9. Steam. VA-11 Hall-A: Cyberpunk Bartender Action. URL: https://store.steampowered.com/app/447530/VA11_Hall_A_Cyberpunk_Bartender_Action/ (дата обращения: 03.04.2026). 10. Steam. Coffee Talk. URL: https://store.steampowered.com/app/914800/Coffee_Talk/ (дата обращения: 03.04.2026).

11. Steam. Yunyun Syndrome!? Rhythm Psychosis. URL: <https://store.steampowered.com/app/2914150> (дата обращения: 03.04.2026).
12. Steam. Glisynth. URL: <https://store.steampowered.com/app/3880410/Glisynth/> (дата обращения: 03.04.2026).
13. Godot Engine. Godot Docs – 4.6 branch (stable). URL: <https://docs.godotengine.org/en/stable/> (дата обращения: 03.03.2026).
14. ArrowVortex. URL: <https://arrowvortex.ddrnl.com/> (дата обращения: 21.03.2026).
15. Aseprite. Animated sprite editor & pixel art tool. URL: <https://www.aseprite.org/> (дата обращения: 03.03.2026).
16. ChatGPT. The ChatGPT home page. URL: <https://chatgpt.com/> (дата обращения: 03.03.2026).
17. Leonardo.Ai. Leonardo.Ai App. URL: <https://app.leonardo.ai/> (дата обращения: 06.03.2026).
18. Suno. AI Music Generator. URL: <https://suno.com/> (дата обращения: 21.03.2026).
19. Audacity. Downloading & installing Audacity. URL: <https://www.audacityteam.org/> (дата обращения: 21.03.2026).
20. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: https://www.youtube.com/watch?v=eEZvcG_TIHo&t=1054s (дата обращения: 22.03.2026).
21. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: https://www.youtube.com/watch?v=_FRiPPbJsFQ&t=269s (дата обращения: 23.03.2026).
22. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: <https://www.youtube.com/watch?v=owbRRxrIp14> (дата обращения: 24.03.2026).
23. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: <https://www.youtube.com/watch?v=JPBmrUsSeok&t=87s> (дата обращения: 25.03.2026).

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист	
Изм.	Лист	№ докум.	Подп.	Дата						54	

24. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: <https://www.youtube.com/watch?v=rnQiyzL2BLk> (дата обращения: 26.03.2026).
25. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: <https://www.youtube.com/watch?v=cZzf1FQQFA0&t=956s> (дата обращения: 27.03.2026).
26. YouTube. Видеоматериал по проектированию алгоритмов и логики ритм-игры. URL: <https://www.youtube.com/watch?v=9XcLoEVnjrA&t=126s> (дата обращения: 28.03.2026).

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										55
Изм.	Лист	№ докум.	Подп.	Дата						

Приложение А
(Справочное)
Экранные формы



Рисунок А.1 – Экран главного меню игры «Сирра»



Рисунок А.2 – Экран пролога (вступительная сцена)

Инв. № подл.					
	Подп. и дата				
	Инв. № дубл.				
Инв. № инв.					
	Взам. инв. №				
	Подп. и дата				
Инв. № подл.					
	Изм				
	Лист				
		№ докум.	Подп.	Дата	

ТПЖА.090301.957 ПЗ

Лист

56

Приложение Б
(Обязательное)

Листинги исходного кода программных модулей

Листинг модуля главного меню – menu.gd

```
1 extends Node2D
2
3 const SETTINGS_OVERLAY = preload("res://Menu/Scripts/Settings/
  settings_overlay.gd")
4 const PROLOGUE_SCENE_PATH := "res://Novel/Scenes/Prologue.tscn"
5
6 const MENU_IDLE_COLOR := Color("#472c2b")
7 const MENU_HOVER_COLOR := Color("#463438")
8 const MENU_HOVER_SCALE := Vector2(1.05, 1.05)
9 const MENU_IDLE_SCALE := Vector2.ONE
10
11 var lights: Array[Sprite2D] = []
12 var base_alphas: Array[float] = []
13 var ui_tweens: Dictionary = {}
14
15 @onready var game_start: Label = $GameStart
16 @onready var game_settings: Label = $GameSettings
17 @onready var game_exit: Label = $GameExit
18 @onready var game_continue: Label = $GameContinue
19
20 func _ready() -> void:
21     cache_lights()
22     setup_label(game_start)
23     setup_label(game_settings)
24     setup_label(game_exit)
25     setup_label(game_continue)
26     update_continue_visibility()
27
28 ## Collect menu lights
29 func cache_lights() -> void:
```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Лист
59

```

30     for node in get_children():
31         if (String(node.name).begins_with("Light")):
32             var sprite := node as Sprite2D
33             lights.append(sprite)
34             base_alphas.append(sprite.modulate.a)
35
36 ## Bind hover and click
37 func setup_label(label: Label) -> void:
38     label.mouse_filter = Control.MOUSE_FILTER_STOP
39     label.add_theme_color_override("font_color", MENU_IDLE_COLOR)
40
41     label.mouse_entered.connect(func() -> void:
42         label.add_theme_color_override("font_color", MENU_HOVER_COLOR)
43         animate_label(label, MENU_HOVER_SCALE, 0.12)
44     )
45     label.mouse_exited.connect(func() -> void:
46         label.add_theme_color_override("font_color", MENU_IDLE_COLOR)
47         animate_label(label, MENU_IDLE_SCALE, 0.12)
48     )
49     label.gui_input.connect(func(event: InputEvent) -> void:
50         if event is InputEventMouseButton and event.button_index ==
51             MOUSE_BUTTON_LEFT and event.pressed:
52             on_label_pressed(label)
53     )
54 ## Handle menu click
55 func on_label_pressed(label: Label) -> void:
56     animate_press(label)
57     if label == game_start:
58         Global.can_resume = true
59         Global.resume_scene_path = PROLOGUE_SCENE_PATH
60         get_tree().change_scene_to_file(PROLOGUE_SCENE_PATH)
61     elif label == game_continue:
62         if Global.can_resume and not Global.resume_scene_path.is_empty():
63             get_tree().change_scene_to_file(Global.resume_scene_path)
64     elif label == game_settings:
65         SETTINGS_OVERLAY.open_for(self)
66     elif label == game_exit:
67         get_tree().quit()
68
69 ## Show continue only when runtime progress exists

```

[illegible]

```
70 func update_continue_visibility() -> void:
71     var can_show: bool = Global.can_resume and not Global.resume_scene_path.
        is_empty()
72     game_continue.visible = can_show
73     game_continue.mouse_filter = Control.MOUSE_FILTER_STOP if can_show else
        Control.MOUSE_FILTER_IGNORE
74
75 ## Smooth label scale
76 func animate_label(label: Label, target_scale: Vector2, duration: float)
    -> void:
77     var t: Tween = create_tween()
78     t.tween_property(label, "scale", target_scale, duration).set_trans(Tween
        .TRANS_SINE).set_ease(Tween.EASE_IN_OUT)
79
80 ## Short click pop
81 func animate_press(label: Label) -> void:
82     var t: Tween = create_tween()
83     t.tween_property(label, "scale", Vector2(0.97, 0.97), 0.06).set_trans(
        Tween.TRANS_SINE).set_ease(Tween.EASE_OUT)
84     t.tween_property(label, "scale", MENU_HOVER_SCALE, 0.12).set_trans(Tween
        .TRANS_SINE).set_ease(Tween.EASE_OUT)
```

Листинг подмодуля настроек – settings_overlay.gd

```
1 extends CanvasLayer
2 class_name SettingsOverlay
3
4 const SETTINGS_SCENE: PackedScene = preload("res://Menu/Scenes/Settings.
    tscn")
5 const DEFAULT_MUSIC_VOLUME := 0.75
6 const MASTER_BUS := "Master"
7
8 static var music_volume: float = DEFAULT_MUSIC_VOLUME    # Real saved value
9 var draft_music_volume: float = DEFAULT_MUSIC_VOLUME     # Temp value
    before save
10
11 @onready var save_button: Button = $Panel/Content/ButtonsRow/SaveButton
```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										61
					Изм	Лист	№ докум.	Подп.	Дата	

```
12 @onready var close_button: Button = $Panel/Content/ButtonsRow/CloseButton
13 @onready var music_slider: HSlider = $Panel/Content/MusicRow/MusicSlider
14 @onready var music_value_label: Label = $Panel/Content/MusicRow/MusicValue
15
16
17 ## Open settings over current scene
18 ## [scene_root] - Any node from active scene
19 static func open_for(scene_root: Node) -> SettingsOverlay:
20     var current_scene: Node = scene_root.get_tree().current_scene
21     var overlay: SettingsOverlay = SETTINGS_SCENE.instantiate()
22     current_scene.add_child(overlay)
23     return overlay
24
25 ## Prepare settings and bind buttons
26 func _ready() -> void:
27     draft_music_volume = music_volume
28     music_slider.value = draft_music_volume
29
30     update_music_label(draft_music_volume)
31     apply_music_volume(music_volume)
32
33     music_slider.value_changed.connect(on_music_changed)
34     save_button.pressed.connect(save_settings)
35     close_button.pressed.connect(queue_free)
36
37 ## Close by Esc
38 func _unhandled_input(event: InputEvent) -> void:
39     if (event.is_action_pressed("ui_cancel")):
40         queue_free()
41
42 ## Update temp value while dragging
43 ## [value] - Slider value 0..1
44 func on_music_changed(value: float) -> void:
45     draft_music_volume = clamp(value, 0.0, 1.0)
46     update_music_label(draft_music_volume)
47
48 ## Show music percent
49 ## [value] - Slider value 0..1
50 func update_music_label(value: float) -> void:
51     var percent: int = int(round(value * 100.0))
52     music_value_label.text = str(percent) + "%"
```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

```

53
54 ## Apply saved volume
55 ## [value] - Volume
56 func apply_music_volume(value: float) -> void:
57     var db: float = linear_to_db(max(value, 0.0001))
58     var bus_index: int = AudioServer.get_bus_index(MASTER_BUS)
59     if (bus_index >= 0):
60         AudioServer.set_bus_volume_db(bus_index, db)
61     for node: Node in get_tree().current_scene.find_children("*, "
        AudioStreamPlayer2D", true, false):
62         node.set("volume_db", db)
63
64 ## Save draft value and apply it
65 func save_settings() -> void:
66     music_volume = draft_music_volume
67     apply_music_volume(music_volume)
68     queue_free()

```

Листинг подмодуля паузы – pause_overlay.gd

```

1 extends CanvasLayer
2 class_name PauseOverlay
3
4 const SETTINGS_OVERLAY = preload("res://Menu/Scripts/Settings/
    settings_overlay.gd")
5 const MENU_SCENE_PATH := "res://Menu/Scenes/Menu.tscn"
6 const OVERLAY_SCENE: PackedScene = preload("res://Menu/Scenes/PauseOverlay
    .tscn")
7
8 @onready var continue_button: Button = $Panel/Content/Buttons/
    ContinueButton
9 @onready var settings_button: Button = $Panel/Content/Buttons/
    SettingsButton
10 @onready var menu_button: Button = $Panel/Content/Buttons/MenuButton
11 @onready var exit_button: Button = $Panel/Content/Buttons/ExitButton
12
13 var resume_scene_path: String = ""

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										63
Изм.	Лист	№ докум.	Подп.	Дата						Копировал
										Формат А4


```

17 @onready var skip_button: Button = $SkipButton
18
19 ## Start prologue lines
20 func _ready() -> void:
21     Global.can_resume = true
22     Global.resume_scene_path = PROLOGUE_SCENE_PATH
23     setup_scene_fade()
24     show_line()
25     skip_button.pressed.connect(skip_prologue)
26
27 ## Go next by space or click
28 func _unhandled_input(event: InputEvent) -> void:
29     if (event is InputEventKey and event.pressed and event.keycode ==
        KEY_SPACE):
30         next_line()
31     if (event is InputEventMouseButton and event.pressed and event.
        button_index == MOUSE_BUTTON_LEFT):
32         next_line()
33
34 ## Show current line
35 func show_line() -> void:
36     var line: Dictionary = lines[current_line]
37     role_label.text = str(line["role"])
38     dialog_label.text = str(line["text"])
39
40 ## Move dialog forward
41 func next_line() -> void:
42     current_line += 1
43     if (current_line >= lines.size()):
44         skip_prologue()
45         return
46     show_line()
47
48
49 ## Skip prologue
50 func skip_prologue() -> void:
51     switch_scene_with_fade(RHYTHM_GAME_SCENE_PATH)
52
53
54 ## Build fade overlay
55 func setup_scene_fade() -> void:

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										66
Изм	Лист	№ докум.	Подп.	Дата						

```

56     fade_layer = CanvasLayer.new()
57     fade_layer.layer = 200
58     add_child(fade_layer)
59
60     fade_color = ColorRect.new()
61     fade_color.set_anchors_preset(Control.PRESET_FULL_RECT)
62     fade_color.color = Color(0, 0, 0, 0)
63     fade_color.mouse_filter = Control.MOUSE_FILTER_IGNORE
64     fade_layer.add_child(fade_color)
65
66
67 ## Fade and open scene
68 ## [scene_path] - Target scene path
69 func switch_scene_with_fade(scene_path: String) -> void:
70     var t: Tween = create_tween()
71     t.tween_property(fade_color, "color:a", 1.0, SCENE_FADE_TIME).set_trans(
72         Tween.TRANS_SINE).set_ease(Tween.EASE_IN_OUT)
73     t.finished.connect(func() -> void:
74         get_tree().change_scene_to_file(scene_path)
75     )

```

Листинг модуля основной новеллы – novel.gd

```

1 extends Node2D
2
3 const RHYTHM_GAME_SCENE_PATH := "res://RhythmGame/Scenes/RhythmGame.tscn"
4 const ENDING_SCENE_PATH := "res://Novel/Scenes/Ending.tscn"
5 const SCENE_FADE_TIME := 0.33
6 const NOVEL_SCENE_PATH := "res://Novel/Scenes/MainNovel.tscn"
7 const PAUSE_OVERLAY = preload("res://Menu/Scripts/Pause/pause_overlay.gd")
8
9 var current_line: int = 0
10 var lines: Array = []
11 var fade_layer: CanvasLayer
12 var fade_color: ColorRect
13
14 const DIALOGS = preload("res://Novel/Data/Dialog/dialogs.gd")

```

```

15
16 @onready var role_label: Label = $DialogPanel/Role
17 @onready var dialog_label: Label = $DialogPanel/DialogText
18 @onready var skip_button: Button = $SkipButton
19
20 @onready var mary: AnimatedSprite2D = $Mary
21 @onready var mary_with_coffee: AnimatedSprite2D = $MaryWithCoffee
22 @onready var kori: AnimatedSprite2D = $Kori
23 @onready var kori_with_coffee: AnimatedSprite2D = $KoriWithCoffee
24 @onready var rose: AnimatedSprite2D = $Rose
25 @onready var rose_with_coffee: AnimatedSprite2D = $RoseWithCoffee
26
27
28 ## Prepare dialog phase
29 func _ready() -> void:
30     Global.can_resume = true
31     Global.resume_scene_path = NOVEL_SCENE_PATH
32     setup_scene_fade()
33     setup_story_phase()
34     show_line()
35     skip_button.pressed.connect(skip_scene)
36
37 ## Go next by space or click
38 func _unhandled_input(event: InputEvent) -> void:
39     if event.is_action_pressed("ui_cancel"):
40         PAUSE_OVERLAY.open_for(self, NOVEL_SCENE_PATH)
41         return
42     if (event is InputEventKey and event.pressed and event.keycode ==
        KEY_SPACE):
43         next_line()
44     if (event is InputEventMouseButton and event.pressed and event.
        button_index == MOUSE_BUTTON_LEFT):
45         next_line()
46
47 ## Pick scene by state
48 func setup_story_phase() -> void:
49     current_line = 0
50     set_all_sprites_hidden()
51
52     match Global.novel_state:
53         "mary_result":

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										68
Изм	Лист	№ докум.	Подп.	Дата						

```

54     mary_with_coffee.visible = true
55     if (Global.last_accuracy >= 50.0):
56         lines = DIALOGS.LINES["mary_happy"]
57         mary_with_coffee.frame = 0
58     else:
59         lines = DIALOGS.LINES["mary_sad"]
60         mary_with_coffee.frame = 0
61 "cory_intro":
62     kori.visible = true
63     kori.frame = 0
64     lines = DIALOGS.LINES["cory_intro"]
65 "cory_result":
66     kori_with_coffee.visible = true
67     if (Global.last_accuracy >= 50.0):
68         lines = DIALOGS.LINES["cory_happy"]
69         kori_with_coffee.frame = 0
70     else:
71         lines = DIALOGS.LINES["cory_sad"]
72         kori_with_coffee.frame = 0
73 "rose_intro":
74     rose.visible = true
75     rose.frame = 0
76     lines = DIALOGS.LINES["rose_intro"]
77 "rose_result":
78     if (Global.bad_results_count >= 2):
79         rose_with_coffee.visible = true
80         rose_with_coffee.frame = 0
81         lines = DIALOGS.LINES["rose_bad_ending"]
82         return
83     rose_with_coffee.visible = true
84     if (Global.last_accuracy >= 50.0):
85         lines = DIALOGS.LINES["rose_happy"]
86         rose_with_coffee.frame = 0
87     else:
88         lines = DIALOGS.LINES["rose_sad"]
89         rose_with_coffee.frame = 0
90 _:
91     mary.visible = true
92     mary.frame = 0
93     lines = DIALOGS.LINES["mary_intro"]
94

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Изм	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ	Лист
											69

```

95 ## Hide all character sprites
96 func set_all_sprites_hidden() -> void:
97     mary.visible = false
98     mary_with_coffee.visible = false
99     kori.visible = false
100    kori_with_coffee.visible = false
101    rose.visible = false
102    rose_with_coffee.visible = false
103
104 ## Show current line
105 func show_line() -> void:
106     var line: Dictionary = lines[current_line]
107     role_label.text = str(line["role"])
108     dialog_label.text = str(line["text"])
109
110 ## Move dialog forward
111 func next_line() -> void:
112     current_line += 1
113     if (current_line >= lines.size()):
114         skip_scene()
115         return
116     show_line()
117
118 ## Route by current novel phase
119 func skip_scene() -> void:
120     match Global.novel_state:
121         "mary_intro":
122             Global.current_client = "mary"
123             switch_scene_with_fade(RHYTHM_GAME_SCENE_PATH)
124         "mary_result":
125             Global.novel_state = "cory_intro"
126             reload_scene_with_fade()
127         "cory_intro":
128             Global.current_client = "boy"
129             switch_scene_with_fade(RHYTHM_GAME_SCENE_PATH)
130         "cory_result":
131             Global.novel_state = "rose_intro"
132             reload_scene_with_fade()
133         "rose_intro":
134             Global.current_client = "hunter"
135             switch_scene_with_fade(RHYTHM_GAME_SCENE_PATH)

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										70
Изм.	Лист	№ докум.	Подп.	Дата						

```
136     "rose_result":
137         Global.novel_state = "mary_intro"
138         switch_scene_with_fade(ENDING_SCENE_PATH)
139     _:
140         Global.current_client = "mary"
141         switch_scene_with_fade(RHYTHM_GAME_SCENE_PATH)
142
143 ## Build fade overlay
144 func setup_scene_fade() -> void:
145     fade_layer = CanvasLayer.new()
146     fade_layer.layer = 200
147     add_child(fade_layer)
148
149     fade_color = ColorRect.new()
150     fade_color.set_anchors_preset(Control.PRESET_FULL_RECT)
151     fade_color.color = Color(0, 0, 0, 0)
152     fade_color.mouse_filter = Control.MOUSE_FILTER_IGNORE
153     fade_layer.add_child(fade_color)
154
155 ## Fade and open scene
156 ## [scene_path] - Target scene path
157 func switch_scene_with_fade(scene_path: String) -> void:
158     var t: Tween = create_tween()
159     t.tween_property(fade_color, "color:a", 1.0, SCENE_FADE_TIME).set_trans(
160         Tween.TRANS_SINE).set_ease(Tween.EASE_IN_OUT)
161     t.finished.connect(func() -> void:
162         get_tree().change_scene_to_file(scene_path)
163     )
164
165 ## Fade and reload scene
166 func reload_scene_with_fade() -> void:
167     var t: Tween = create_tween()
168     t.tween_property(fade_color, "color:a", 1.0, SCENE_FADE_TIME).set_trans(
169         Tween.TRANS_SINE).set_ease(Tween.EASE_IN_OUT)
170     t.finished.connect(func() -> void:
171         get_tree().reload_current_scene()
172     )
```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Листинг модуля концовок – ending.gd

```

1 extends Node2D
2
3 const MENU_SCENE_PATH := "res://Menu/Scenes/Menu.tscn"
4 const SCENE_FADE_TIME := 0.33
5 const ENDING_LINES = preload("res://Novel/Data/Dialog/ending_lines.gd")
6
7 @onready var good_layer: Node2D = $GoodEnding
8 @onready var bad_layer: ColorRect = $BadEnding
9 @onready var role_label: Label = $Role
10 @onready var text_label: Label = $DialogText
11 @onready var continue_button: Button = $ContinueButton
12
13 var fade_layer: CanvasLayer
14 var fade_color: ColorRect
15 var current_line: int = 0
16 var lines: Array = []
17
18
19 ## Prepare ending by result
20 func _ready() -> void:
21     Global.can_resume = false
22     Global.resume_scene_path = ""
23     setup_scene_fade()
24     setup_ending()
25     show_line()
26     continue_button.visible = false
27     continue_button.pressed.connect(go_to_menu)
28
29 ## Go next by space or click
30 func _unhandled_input(event: InputEvent) -> void:
31     if (event is InputEventKey and event.pressed and event.keycode ==
        KEY_SPACE):
32         next_line()
33     if (event is InputEventMouseButton and event.pressed and event.
        button_index == MOUSE_BUTTON_LEFT):
34         next_line()
35

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										72
Изм	Лист	№ докум.	Подп.	Дата						Копировал
										Формат А4


```

36 ## Pick ending variant
37 func setup_ending() -> void:
38     var is_good: bool = Global.bad_results_count < 2
39     good_layer.visible = is_good
40     bad_layer.visible = not is_good
41     current_line = 0
42
43     if is_good:
44         lines = ENDING_LINES.GOOD
45     else:
46         lines = ENDING_LINES.BAD
47
48
49 ## Show current ending line
50 func show_line() -> void:
51     if lines.is_empty():
52         return
53     var line: Dictionary = lines[current_line]
54     role_label.text = str(line["role"])
55     text_label.text = str(line["text"])
56
57
58 ## Move ending dialog forward
59 func next_line() -> void:
60     if lines.is_empty():
61         go_to_menu()
62         return
63     if (current_line >= lines.size() - 1):
64         continue_button.visible = true
65         return
66     current_line += 1
67     show_line()
68
69
70 ## Return to menu and reset state
71 func go_to_menu() -> void:
72     Global.novel_state = "mary_intro"
73     Global.bad_results_count = 0
74     switch_scene_with_fade(MENU_SCENE_PATH)
75
76

```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм.	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Лист

73

```

77 ## Build fade overlay
78 func setup_scene_fade() -> void:
79     fade_layer = CanvasLayer.new()
80     fade_layer.layer = 200
81     add_child(fade_layer)
82
83     fade_color = ColorRect.new()
84     fade_color.set_anchors_preset(Control.PRESET_FULL_RECT)
85     fade_color.color = Color(0, 0, 0, 0)
86     fade_color.mouse_filter = Control.MOUSE_FILTER_IGNORE
87     fade_layer.add_child(fade_color)
88
89
90 ## Fade and open scene
91 ## [scene_path] - Target scene path
92 func switch_scene_with_fade(scene_path: String) -> void:
93     var t: Tween = create_tween()
94     t.tween_property(fade_color, "color:a", 1.0, SCENE_FADE_TIME).set_trans(
        Tween.TRANS_SINE).set_ease(Tween.EASE_IN_OUT)
95     t.finished.connect(func() -> void:
96         get_tree().change_scene_to_file(scene_path)
97     )

```

Листинг координирующего модуля ритм-игры – rhythm_game.gd

```
1 extends Node2D
2
3 const SM_PARSER = preload("res://RhythmGame/Scripts/Parsing/sm_parser.gd")
4 const SIDE_LIGHTS_CONTROLLER = preload("res://RhythmGame/Scripts/UI/
    side_lights_controller.gd")
5 const SONGS_LIBRARY_DATA = preload("res://RhythmGame/Scripts/State/
    songs_library.gd")
6 const POP_UI = preload("res://RhythmGame/Scripts/UI/pop_effect.gd")
7 const CONTROLS_HINT_SCENE = preload("res://Novel/Scenes/ControlsHint.tscn
    ")
8 const PAUSE_OVERLAY = preload("res://Menu/Scripts/Pause/pause_overlay.gd")
9 const RHYTHM_SCENE_PATH := "res://RhythmGame/Scenes/RhythmGame.tscn"
```

```

10
11 @export var songs_library = SONGS_LIBRARY_DATA.SONGS_LIBRARY
12
13 @onready var audio = $SongPlayer
14 @onready var spawns = [$SpawnPoint1, $SpawnPoint2, $SpawnPoint3,
    $SpawnPoint4]
15 @onready var light_left: Sprite2D = $LightLeft
16 @onready var light_right: Sprite2D = $LightRight
17 @onready var combo_label: Label = $Combo
18 @onready var combo_score_label: Label = $ComboScore
19 @onready var combo_text_label: Label = $ComboText
20
21 var note_scene = preload("res://RhythmGame/Scenes/Note.tscn")
22 var hold_scene = preload("res://RhythmGame/Scenes/LongNote.tscn")
23
24 const MAX_NOTE_SCORE: int = 10      # Base score per note
25 const BEATS_TO_TARGET: int = 6      # Travel time in beats (control note
    speed)
26 const PROGRESS_SMOOTH_SPEED: float = 3.2
27 const COMBO_UI_FADE_SPEED: float = 6.0
28 const SCENE_FADE_TIME := 0.33
29
30 var bpm: float = 0.0
31 var beat: float = 0.0
32 var note_speed: float = 0.0        # (Distance / travel_time)
33 var travel_time: float = 0.0      # Time to reach judge
34 var audio_started: bool = false
35
36 var game_time: float = 0.0        # Global timer for sync
37 var is_playing: bool = false      # Pause/stop
38
39 var all_notes_data: Array = []
40 var song_key: String = "crimson_pulse"
41 var progress_value: float = 100.0
42 var light_left_base_alpha: float = 0.0
43 var light_right_base_alpha: float = 0.0
44 var last_combo_score: int = 0
45 var combo_ui_alpha: float = 0.0
46
47 var round_finished: bool = false
48 var intro_waiting: bool = false

```

[illegible]

```

49 var pending_song_data: Dictionary = {}
50 var controls_hint_overlay: CanvasLayer = null
51
52 var side_lights_controller = SIDE_LIGHTS_CONTROLLER.new()
53 var pop_controller = POP_UI.new()
54 var fade_layer: CanvasLayer
55 var fade_rect: ColorRect
56
57 ## Initialize game
58 func _ready() -> void:
59     Global.can_resume = true
60     Global.resume_scene_path = RHYTHM_SCENE_PATH
61     setup_scene_fade()
62     reset_round_state()
63     song_key = get_song_key_for_client()
64     var data: Dictionary = songs_library[song_key]
65     bpm = data["bpm"]
66
67     ## Math
68     beat = 60.0 / bpm
69     travel_time = beat * BEATS_TO_TARGET
70     var distance: float = abs(spawns[0].global_position.y - $JudgmentRoot.
        global_position.y)
71     note_speed = distance / travel_time
72
73     all_notes_data = SM_PARSER.full_parser(data["path_sm"], beat, data["
        offset"])    # Parsing
74
75     $ProgressBar.modulate = Color(1.0, 1.0, 1.0, 0.82)
76     $ProgressBar.value = 100.0
77     light_left_base_alpha = light_left.modulate.a
78     light_right_base_alpha = light_right.modulate.a
79     round_finished = false
80
81     if Global.rhythm_controls_seen:
82         start_game(data)
83     else:
84         pending_song_data = data
85         show_controls_hint_once()
86
87 func _unhandled_input(event: InputEvent) -> void:

```

Изм.	Лист	№ докум.	Подп.	Дата
Интв. № подл.	Подп. и дата	Взам. инв. №	Интв. № дубл.	Подп. и дата

ТПЖА.090301.957 ПЗ

```

88     if event.is_action_pressed("ui_cancel"):
89         PAUSE_OVERLAY.open_for(self, RHYTHM_SCENE_PATH)
90
91     ## Start game
92     ## [song_data] - dictionary with all data of songs to take song name
93     func start_game(song_data: Dictionary) -> void:
94         audio.stream = song_data["song"]
95         game_time -= travel_time
96         is_playing = true
97         audio_started = false                # To play music after offset to
            hit in the beat
98
99     ## Spawn note
100    ## [lag] - delay with computer draw and current time
101    ## [line_index] - note lane
102    func spawn_note(lag: float, line_index: int) -> void:
103        var note = note_scene.instantiate()
104        var spawn_node = spawns[line_index]
105        note.global_position = spawn_node.global_position
106        note.global_position.y += lag*note_speed
107        note.speed = note_speed
108        note.line_index = line_index
109        add_child(note)
110
111    ## Spawn hold
112    ## [lag] - delay with computer draw and current time
113    ## [line_index] - hold lane
114    func spawn_hold_note(lag: float, line_index: int, duration: float) -> void
        :
115        var hold = hold_scene.instantiate()
116        var spawn_node = spawns[line_index]
117        hold.global_position = spawn_node.global_position
118        hold.global_position.y += lag * note_speed
119        hold.speed = note_speed
120        hold.line_index = line_index
121        hold.judgment_y = $JudgmentRoot.global_position.y
122        add_child(hold)
123        var length = duration * note_speed
124        hold.setup_hold(length)
125
126    ## Main game update

```

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Лист

77

```

127 func _process(delta: float) -> void:
128     if intro_waiting:
129         update_back(delta)
130         update_lights(delta)
131         return
132
133     update_progress_bar(delta)
134     update_ui(delta)
135
136     # Boost background when combo is high
137     update_back(delta)
138     update_lights(delta)
139     update_combo_pop()
140
141     game_time += delta
142     var current_sync_time: float = get_current_sync_time()
143     spawn_due_notes(current_sync_time)
144     try_start_audio()
145     try_finish_round()
146
147 ## Updates score and combo labels
148 func update_ui(delta: float) -> void:
149     $Score.text = str(Global.score)
150     combo_text_label.text = Global.combo
151     combo_score_label.text = str(Global.combo_score)
152
153     # Smooth show/hide combo UI
154     var has_combo: bool = (Global.combo_score > 0)
155     var target_alpha: float = 1.0 if has_combo else 0.0
156     combo_ui_alpha = lerp(combo_ui_alpha, target_alpha, min(delta *
        COMBO_UI_FADE_SPEED, 1.0))
157
158     combo_label.visible = true
159     combo_text_label.visible = true
160     combo_score_label.visible = true
161     combo_label.modulate.a = combo_ui_alpha
162     combo_text_label.modulate.a = combo_ui_alpha
163     combo_score_label.modulate.a = combo_ui_alpha
164
165 ## Updates background speed and blackout
166 func update_back(delta) -> void:

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ				Лист
									78
Изм.	Лист	№ докум.	Подп.	Дата					


```

238     return "moonfall"
239     return song_key
240
241
242 ## Finish round when song and notes are done
243 func try_finish_round() -> void:
244     if round_finished:
245         return
246     if not audio_started:
247         return
248     if audio.playing:
249         return
250     if not all_notes_data.is_empty():
251         return
252     if not get_tree().get_nodes_in_group("notes").is_empty():
253         return
254
255     round_finished = true
256     Global.last_accuracy = get_current_accuracy()
257     if (Global.last_accuracy < 50.0):
258         Global.bad_results_count += 1
259     Global.novel_state = get_result_state_for_client()
260     switch_scene_with_fade("res://Novel/Scenes/MainNovel.tscn")
261
262
263 ## Get next novel result phase
264 func get_result_state_for_client() -> String:
265     if (Global.current_client == "boy"):
266         return "cory_result"
267     if (Global.current_client == "hunter"):
268         return "rose_result"
269     return "mary_result"
270
271 ## Manage progress bar
272 func update_progress_bar(delta: float) -> void:
273     if (Global.judged_count <= 0):
274         $ProgressBar.value = 100
275         return
276     var max_score_so_far: float = float(Global.judged_count * MAX_NOTE_SCORE
277         )
277     var accuracy: float = (float(Global.score) / max_score_so_far) * 100.0

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ				Лист
									81
Изм.	Лист	№ докум.	Подп.	Дата					

```

278 var target_value: float = clamp(accuracy, 0.0, 100.0)
279 progress_value = lerp(progress_value, target_value, min(delta * 6.0,
    1.0))
280 $ProgressBar.value = progress_value
281
282
283 ## Get accuracy by judged notes
284 func get_current_accuracy() -> float:
285     if (Global.judged_count <= 0):
286         return 0.0
287     var max_score_so_far: float = float(Global.judged_count * MAX_NOTE_SCORE
    )
288     return clamp((float(Global.score) / max_score_so_far) * 100.0, 0.0,
    100.0)
289
290
291 ## Build fade overlay
292 func setup_scene_fade() -> void:
293     fade_layer = CanvasLayer.new()
294     fade_layer.layer = 200
295     add_child(fade_layer)
296
297     fade_rect = ColorRect.new()
298     fade_rect.set_anchors_preset(Control.PRESET_FULL_RECT)
299     fade_rect.color = Color(0, 0, 0, 0)
300     fade_rect.mouse_filter = Control.MOUSE_FILTER_IGNORE
301     fade_layer.add_child(fade_rect)
302
303
304 ## Fade and open scene
305 ## [scene_path] - Target scene path
306 func switch_scene_with_fade(scene_path: String) -> void:
307     var t: Tween = create_tween()
308     t.tween_property(fade_rect, "color:a", 1.0, SCENE_FADE_TIME).set_trans(
    Tween.TRANS_SINE).set_ease(Tween.EASE_IN_OUT)
309     t.finished.connect(func() -> void:
310         get_tree().change_scene_to_file(scene_path)
311     )
312
313
314 ## Show controls hint on first launch

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										82
Изм.	Лист	№ докум.	Подп.	Дата						

```
315 func show_controls_hint_once() -> void:
316     intro_waiting = true
317     controls_hint_overlay = CONTROLS_HINT_SCENE.instantiate()
318     add_child(controls_hint_overlay)
319     controls_hint_overlay.confirmed.connect(on_controls_hint_confirmed)
320
321
322 ## Start rhythm after confirm
323 func on_controls_hint_confirmed() -> void:
324     Global.rhythm_controls_seen = true
325     intro_waiting = false
326     start_game(pending_song_data)
```

Листинг подмодуля обработки файлов нот – sm_parser.gd

```
1 extends RefCounted
2
3 ## Parses .sm file
4 ## [path] - Path to .sm
5 ## [beat] - Beat duration in seconds
6 ## [offset] - Song start offset
7 static func full_parser(path: String, beat: float, offset: float) -> Array
8     :
9     var current_time: float = offset
10    var holds: Dictionary = {}          # Keep hold starts by lane
11    var all_notes_data: Array = []      # Save all parsed notes here
12
13    # Prepare parser
14    var sm_file := FileAccess.open(path, FileAccess.READ)
15    if not sm_file:
16        push_error("Cannot open chart file: " + path)
17        return all_notes_data
18    var content: String = sm_file.get_as_text()
19    sm_file.close()
20
21    # Skip
22    var pos: int = content.find("0,0,0,0,0:")
```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

```

22  if pos == -1:
23      push_error("Chart marker not found: " + path)
24      return all_notes_data
25  pos += 10
26
27  # Split into measures
28  var raw_data: String = content.substr(pos)
29  var measures: PackedStringArray = raw_data.replace(";", " ").split(",")
30
31  # Parse each measure
32  for measure in measures:
33      var lines: Array = []
34      for i in measure.strip_edges().split("\n"):
35          if i.length() == 4:  # Keep only 4-lane rows
36              lines.append(i)
37
38          if lines.is_empty():
39              continue
40
41          var time_per_line: float = (beat * 4.0) / float(lines.size()) # One
measure has 4 beats
42
43          # Parse symbols in each lane
44          for line in lines:
45              for j in range(4):
46                  var symbol = line[j]
47                  match symbol:
48                      "1":
49                      all_notes_data.append({"time": current_time, "type": "note", "
lane": j})
50                      "2":
51                      holds[j] = current_time
52                      "3":
53                      if holds.has(j):
54                          var duration: float = current_time - float(holds[j])
55                          all_notes_data.append({"time": float(holds[j]), "type": "
hold", "lane": j, "duration": duration})
56                          holds.erase(j)
57                      current_time += time_per_line  # Move timeline to next row
58
59  # Sort by time for stable spawn order

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										84
Изм	Лист	№ докум.	Подп.	Дата						

```
60 all_notes_data.sort_custom(func(a, b): return a.time < b.time)
61 return all_notes_data
```

Листинг подмодуля нот и их поведения – note.gd

```
1 extends Area2D
2
3 # Delete note after this Y
4 const DESPAWN_Y = 450.0
5
6 var speed: float = 0.0
7 var line_index: int = 0
8
9 # Is this obviously a note or hold
10 var is_hold: bool = false
11
12 func _process(delta: float) -> void:
13     # Move
14     position.y += speed * delta
15     if global_position.y >= DESPAWN_Y:
16         queue_free()
```

Листинг подмодуля нот и их поведения – hold_note.gd

```
1 extends Area2D
2
3 signal hold_feedback(hit_color: Color)
4
5 # Parts of hold note
6 @onready var head = $HeadSprite
7 @onready var body = $Body
8 @onready var bottom = $TailSprite
9
```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

```
10 const SCORE_SERVICE = preload("res://RhythmGame/Scripts/Controllers/
    score_service.gd")
11
12 const MISS_WINDOW_PX: float = 24.0      # Miss window after judgment
13
14 var speed: float = 0.0
15 var line_index: int = 0
16
17 var is_hold: bool = true
18 var is_being_held: bool = false
19 var is_finished: bool = false
20 var is_missed: bool = false
21 var was_started: bool = false
22 var was_scored: bool = false
23
24 var judgment_y: float = 0.0
25 var initial_body_length: float = 0.0  # Starting body length
26
27 func _process(delta: float) -> void:
28     # Move
29     position.y += speed * delta
30
31     # Holding logic
32     if is_being_held:
33         var overshoot = position.y - judgment_y  # How much head is
            compressed
34         head.position.y = lerp(head.position.y, -overshoot, 0.22) # Hold head
            around judgment smoothly
35         var length_to_decrease: float = max(body.size.y - (speed * delta),
            0.0)  # Decrease body
36
37         # Body ended while still holding
38         if length_to_decrease <= 0.0:
39             body.size.y = 0.0
40             apply_hold_result(1.0)
41             is_finished = true
42             is_being_held = false
43         # Body not ended -> decrease
44         else:
45             body.size.y = length_to_decrease
```

Изм	Лист	№ докум.	Подп.	Дата	Подп. и дата
					Инов. № дубл.
					Взам. инв. №
Изм	Лист	№ докум.	Подп.	Дата	Подп. и дата
					Инов. № дубл.
					Взам. инв. №
Изм	Лист	№ докум.	Подп.	Дата	Подп. и дата
					Инов. № дубл.
					Взам. инв. №

ТПЖА.090301.957 ПЗ

```

46     body.position.y = head.position.y - body.size.y    # Hold body around
        judgment/head
47     bottom.position.y = body.position.y
48
49     # Not pressed and never started
50     elif not is_missed and not was_started and global_position.y >
        judgment_y + MISS_WINDOW_PX:
51         mark_as_miss()
52
53     var tail_global_y = bottom.global_position.y
54
55     # Remove hold only after tail passes judgment zone
56     if (is_missed or is_finished or (was_scored and not is_being_held)) and
        tail_global_y > judgment_y + MISS_WINDOW_PX:
57         queue_free()
58
59     ## Setup every part of hold note
60     ## [length_px] - duration of hold note in pixels
61     func setup_hold(length_px: float) -> void:
62         var safe_length = max(length_px, 0.0)
63         initial_body_length = safe_length
64         body.size.y = safe_length
65         head.position.y = 0
66         body.position.y = -safe_length
67         bottom.position.y = -safe_length
68         was_started = false
69         was_scored = false
70
71     ## Check holding
72     func start_holding() -> void:
73         if not is_missed and not is_finished:
74             is_being_held = true
75             was_started = true
76
77     ## Stop holding
78     func stop_holding() -> void:
79         # Ignore after miss or finish
80         if is_missed or is_finished:
81             return
82
83         is_being_held = false

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ				Лист
									87
Изм	Лист	№ докум.	Подп.	Дата					

```

84  apply_hold_result(get_hold_progress())
85
86  ## Gets hold progress ratio from 0 to 1
87  func get_hold_progress() -> float:
88      if (initial_body_length <= 0.0):
89          return 0.0
90      var ratio: float = 1.0 - (body.size.y / initial_body_length)
91      return clamp(ratio, 0.0, 1.0)
92
93  ## Applies hold score by hold progress
94  ## [progress] - Current hold progress ratio
95  func apply_hold_result(progress: float) -> void:
96      var result: Dictionary = SCORE_SERVICE.evaluate_hold_progress(progress)
97      Global.combo = result["combo_text"]
98      Global.score += int(result["score_delta"])
99      Global.combo_score += int(result["combo_delta"])
100     Global.judged_count += 1
101     hold_feedback.emit(result["hit_color"])
102     was_scored = true
103
104  ## Mark miss
105  func mark_as_miss() -> void:
106      is_missed = true
107      is_being_held = false
108      Global.combo = "Miss"
109      Global.combo_score = 0
110      Global.judged_count += 1
111      hold_feedback.emit(SCORE_SERVICE.MISS_COLOR)
112      var tween = create_tween()
113      tween.tween_property(self, "modulate", Color(0.3, 0.3, 0.3, 0.5), 0.3)

```

Листинг подмодуля оценки действий игрока – judgment_lane.gd

```

1  extends Area2D
2
3  @export var assigned_key: note_keys    # Assigned key for this lane
4  @onready var feedback_sprite = self    # Sprite used for hit color feedback

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ					Лист
										88
Изм.	Лист	№ докум.	Подп.	Дата						Копировал
										Формат А4


```

5
6 # Animation tree for start/hold/end states
7 @onready var anim_tree: AnimationTree = find_current_anim()
8
9 # Shared score rules and colors
10 const SCORE_SERVICE = preload("res://RhythmGame/Scripts/Controllers/
    score_service.gd")
11
12 const HOLD_WINDOW: float = 20.0      # Release window near hold end
13 const TAP_FEEDBACK_TIME: float = 0.46
14 const HOLD_FEEDBACK_TIME: float = 0.70
15
16 # Available lane actions
17 enum note_keys {S_NOTE, D_NOTE, K_NOTE, L_NOTE}
18
19 # Playback handle for lane state machine
20 var playback: AnimationNodeStateMachinePlayback
21
22 # Map enum to input action names
23 var key_map = {
24     note_keys.S_NOTE: "S_NOTE",
25     note_keys.D_NOTE: "D_NOTE",
26     note_keys.K_NOTE: "K_NOTE",
27     note_keys.L_NOTE: "L_NOTE",
28 }
29
30 # State names in animation tree
31 var s_start: String
32 var s_hold: String
33 var s_end: String
34
35 # Notes currently inside this judgment area
36 var notes_in_area: Array[Area2D] = []
37
38 var is_pressing: bool = false      # Pressed now
39 var active_hold_note: Area2D = null # Active hold note in this lane
40 var feedback_tween: Tween
41
42 ## Finds animation tree among lane children
43 func find_current_anim() -> AnimationTree:
44     # Look through children and pick first animation tree

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ				Лист
									89
Изм	Лист	№ докум.	Подп.	Дата					

```

45  for child in get_children():
46      if child is AnimationTree:
47          return child
48  return null
49
50 ## Initializes lane animation state names
51 func _ready() -> void:
52     # Build lane-specific animation state names (Start1/2/3/4)
53     var index = str(int(assigned_key)+1)
54     s_start = "Start" + index
55     s_hold = "Hold" + index
56     s_end = "End" + index
57
58     if anim_tree:
59         playback = anim_tree.get("parameters/playback")
60
61 ## Runs lane hold animation while key is pressed
62 func _process(_delta: float) -> void:
63     if is_pressing:
64         # Keep hold animation active while key is held
65         if playback and playback.get_current_node() != s_hold:
66             playback.travel(s_hold)
67
68 ## Handles lane input press/release events
69 func _unhandled_input(event: InputEvent) -> void:
70     # Get action for this lane
71     var action = key_map[assigned_key]
72
73     # Key pressed
74     if event.is_action_pressed(action):
75         # Lane is pressed now
76         is_pressing = true
77         # Play start animation
78         if playback:
79             playback.travel(s_start)
80         # Try to hit note
81         handle_note_hit()
82
83     # Key released
84     if event.is_action_released(action):
85         # Lane is released now

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ	Лист
						90
						Изм

```

86     is_pressing = false
87     # Play end animation
88     if playback:
89         playback.travel(s_end)
90     # Send release to hold note
91     if active_hold_note:
92         break_holding()
93
94 ## Tries to hit the closest note in lane queue
95 func handle_note_hit() -> void:
96     # Take closest note to judgment line
97     var note := get_closest_note()
98     # Note is existing
99     if not is_instance_valid(note):
100         return
101
102     var distance: float = get_note_judgment_distance(note)    # Distance to
        judge
103     var is_hold_note = note.get("is_hold") == true            # Check note type
104
105     var hit_window_px: float = SCORE_SERVICE.LAME_WINDOW_PX    # Wider
106     if (is_hold_note):
107         hit_window_px += HOLD_WINDOW
108
109     # Ignore press if note is outside hit window
110     if (distance > hit_window_px):
111         return
112     if is_hold_note:
113         # Save active hold
114         active_hold_note = note
115         note.start_holding()
116         # Remove processed note
117         notes_in_area.erase(note)
118         return
119
120     # Tap note
121     rating(distance)
122     note.queue_free()
123     # Remove processed note
124     notes_in_area.erase(note)
125

```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм.	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Лист

91

```

126 ## Applies score result by distance
127 ## [distance] - Distance between judgment and note
128 func rating(distance: float) -> void:
129     # Get hit result from score rules
130     var result: Dictionary = SCORE_SERVICE.evaluate_distance(distance)
131     Global.judged_count += 1    # For progress bar
132     Global.combo = result["combo_text"]
133     Global.score += int(result["score_delta"])
134     if (Global.combo == "Miss"):
135         Global.combo_score = 0
136     else:
137         Global.combo_score += int(result["combo_delta"])
138     apply_feedback_color(result["hit_color"], TAP_FEEDBACK_TIME)
139
140 ## Colors lane feedback and fades back
141 ## [color] - Color for this hit result
142 ## [fade_time] - Fade duration in seconds
143 func apply_feedback_color(color: Color, fade_time: float =
    TAP_FEEDBACK_TIME) -> void:
144     if feedback_tween and is_instance_valid(feedback_tween):
145         feedback_tween.kill()
146     feedback_sprite.modulate = color
147     feedback_tween = create_tween()
148     feedback_tween.tween_property(feedback_sprite, "modulate", Color(1.0,
        1.0, 1.0, 1.0), fade_time).set_trans(Tween.TRANS_SINE).set_ease(Tween.
        EASE_IN_OUT)
149
150 ## Handles key release for active hold
151 func break_holding() -> void:
152     # If hold note still exists, tell it key was released
153     if is_instance_valid(active_hold_note):
154         active_hold_note.stop_holding()
155     # Clear active hold reference in lane
156     active_hold_note = null
157
158 ## Applies lane feedback from hold result
159 ## [hit_color] - Hold judgment color
160 func _on_hold_feedback(hit_color: Color) -> void:
161     apply_feedback_color(hit_color, HOLD_FEEDBACK_TIME)
162
163 ## Gets closest note to lane judgment

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ	Лист
						92
Изм	Лист	№ докум.	Подп.	Дата		

```

164 func get_closest_note() -> Area2D:
165     var closest_note: Area2D = null
166     var closest_distance: float = INF
167     for note in notes_in_area:
168         var distance: float = get_note_judgment_distance(note)
169         # Update new closet distance and closest note
170         if (distance < closest_distance):
171             closest_distance = distance
172             closest_note = note
173     return closest_note
174
175 ## Gets note distance to lane judgment
176 func get_note_judgment_distance(note: Area2D) -> float:
177     # For hold use head position, not root
178     if (note.get("is_hold") == true):
179         var head_node: Node = note.get_node_or_null("HeadSprite")
180         if (head_node and head_node is Node2D):
181             return abs(global_position.y - head_node.global_position.y)
182     return abs(global_position.y - note.global_position.y)
183
184 ## Handles note enter
185 func _on_area_entered(area: Area2D) -> void:
186     # Keep only notes from this lane
187     if area.is_in_group("notes") and area.get("line_index") == int(
        assigned_key):
188         # Connect hold feedback early, so hold miss color also reaches lane
189         if (area.get("is_hold") == true):
190             if not area.is_connected("hold_feedback", Callable(self, "
                _on_hold_feedback"))):
191                 area.connect("hold_feedback", Callable(self, "_on_hold_feedback"))
192         # Add note to lane queue
193         notes_in_area.append(area)
194
195 ## Tracks notes leaving area and applies miss when needed
196 func _on_area_exited(area: Area2D) -> void:
197     # Not our note -> skip
198     if not (area in notes_in_area):
199         return
200     notes_in_area.erase(area)
201     # Hold miss is handled in hold script
202     if (area.get("is_hold") == true):

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ	Лист
						93
Изм.	Лист	№ докум.	Подп.	Дата		

```
203     return
204     # Remaining case means a miss
205     Global.combo = "Miss"
206     Global.combo_score = 0
207     Global.judged_count += 1
208     apply_feedback_color(SCORE_SERVICE.MISS_COLOR)
209
210
211
```

Листинг подмодуля оценки действий игрока – score_service.gd

```
1 extends RefCounted
2
3 const PERFECT_WINDOW_PX: float = 50.0
4 const GOOD_WINDOW_PX: float = 80.0
5 const LAME_WINDOW_PX: float = 110.0
6
7 const PERFECT_SCORE: int = 10
8 const GOOD_SCORE: int = 5
9 const LAME_SCORE: int = 1
10 const COMBO_STEP: int = 10
11
12 const PERFECT_COLOR := Color(1.0, 0.0, 0.318, 1.0)
13 const GOOD_COLOR := Color(1.0, 0.664, 0.531, 1.0)
14 const LAME_COLOR := Color(0.336, 0.782, 0.878, 1.0)
15 const MISS_COLOR := Color(0.125, 0.072, 0.146, 1.0)
16
17 static func evaluate_distance(distance: float) -> Dictionary:
18     if distance < PERFECT_WINDOW_PX:
19         return {
20             "combo_text": "Bloody bite",
21             "score_delta": PERFECT_SCORE,
22             "combo_delta": COMBO_STEP,
23             "reset_combo": false,
24             "hit_color": PERFECT_COLOR,
25         }
```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

```

26  if distance < GOOD_WINDOW_PX:
27      return {
28          "combo_text": "Bloody",
29          "score_delta": GOOD_SCORE,
30          "combo_delta": COMBO_STEP,
31          "reset_combo": false,
32          "hit_color": GOOD_COLOR,
33      }
34  if distance < LAME_WINDOW_PX:
35      return {
36          "combo_text": "Lame",
37          "score_delta": LAME_SCORE,
38          "combo_delta": COMBO_STEP,
39          "reset_combo": false,
40          "hit_color": LAME_COLOR,
41      }
42
43  return {
44      "combo_text": "Miss",
45      "score_delta": 0,
46      "combo_delta": 0,
47      "reset_combo": true,
48      "hit_color": MISS_COLOR,
49  }
50
51  static func evaluate_hold_progress(progress: float) -> Dictionary:
52      if progress >= 0.85:
53          return {
54              "combo_text": "Bloody bite",
55              "score_delta": PERFECT_SCORE,
56              "combo_delta": COMBO_STEP,
57              "hit_color": PERFECT_COLOR,
58          }
59      if progress >= 0.50:
60          return {
61              "combo_text": "Bloody",
62              "score_delta": GOOD_SCORE,
63              "combo_delta": COMBO_STEP,
64              "hit_color": GOOD_COLOR,
65          }
66      return {

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Инв. № подл.					Лист	
											95
Изм.	Лист	№ докум.	Подп.	Дата	ТПЖА.090301.957 ПЗ						

```
67     "combo_text": "Lame",
68     "score_delta": LAME_SCORE,
69     "combo_delta": COMBO_STEP,
70     "hit_color": LAME_COLOR,
71 }
```

Листинг подмодуля управления подсветкой ламп —
side_lights_controller.gd

```
1 extends RefCounted
2
3 const BASE_PULSE_SPEED: float = 4.4
4 const BASE_PULSE_AMOUNT: float = 0.18
5 const SECOND_PULSE_SPEED: float = 8.2
6 const SECOND_PULSE_AMOUNT: float = 0.08
7 const COMBO_BOOST_ALPHA: float = 0.40
8 const SMOOTH_SPEED: float = 10.0
9
10 ## Updates both side lights every frame
11 ## [left_light] - Left light sprite
12 ## [right_light] - Right light sprite
13 ## [left_base_alpha] - Base alpha for left light
14 ## [right_base_alpha] - Base alpha for right light
15 ## [song_time] - Current game time
16 ## [combo_score] - Current combo score
17 func update_lights(left_light: Sprite2D, right_light: Sprite2D,
18     left_base_alpha: float, right_base_alpha: float, song_time: float,
19     combo_score: int, delta: float) -> void:
20
21     var pulse_1: float = sin(song_time * BASE_PULSE_SPEED) *
22         BASE_PULSE_AMOUNT
23     var pulse_2: float = sin(song_time * SECOND_PULSE_SPEED) *
24         SECOND_PULSE_AMOUNT
25     var pulse: float = pulse_1 + pulse_2
26
27     # Add brightness on high combo
28     var boost: float = 0.0
```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ

Лист

96

Листинг подмодуля библиотеки композиций – `songs library.gd`

ТПЖА.090301.957 ПЗ


```
11
12 # Judged notes for accuracy percent
13 var judged_count: int = 0
14
15 # Current novel client key: mary/boy/hunter
16 var current_client: String = "mary"
17
18 # Novel flow state: mary_intro -> mary_result
19 var novel_state: String = "mary_intro"
20
21 # Last finished rhythm accuracy in percent
22 var last_accuracy: float = 0.0
23
24 # Bad rhythm results for ending logic
25 var bad_results_count: int = 0
26
27 # Show controls hint only once before first rhythm start
28 var rhythm_controls_seen: bool = false
29
30 # Runtime-only continue from menu (no file save)
31 var can_resume: bool = false
32 var resume_scene_path: String = ""
```

Листинг общего UI-компонента фоновой анимации
floating_shapes_layer.gd

```
1 extends Node2D
2
3 @export var shape_texture: Texture2D
4 @export var shapes_count: int = 35
5 @export var min_scale: float = 0.75
6 @export var max_scale: float = 1.85
7 @export var min_speed_y: float = 18.0
8 @export var max_speed_y: float = 58.0
9 @export var drift_x: float = 14.0
10 @export var pulse_strength: float = 0.22
11 @export var pulse_speed: float = 2.2
```

Подп. и дата		ТПЖА.090301.957 ПЗ				Лист
Инв. № дубл.						99
Взам. инв. №						
Подп. и дата						
Инв. № подл.	Изм	Лист	№ докум.	Подп.	Дата	

```

12
13 # Rendered all sprites
14 var sprites: Array[Sprite2D] = []
15 # Speed (x/y random flight)
16 var speed: Array[Vector2] = []
17 # Random for alpha pulse
18 var pulse_offsets: Array[float] = []
19 # Base alpha for each particle
20 var base_alphas: Array[float] = []
21
22 ## Builds floating clones once
23 func _ready() -> void:
24     randomize()
25     # Work in screen-centered coordinates
26     var viewport_size: Vector2 = get_viewport_rect().size
27     var half_w: float = viewport_size.x * 0.5
28     var half_h: float = viewport_size.y * 0.5
29
30     var texture := shape_texture
31
32     # Create particles with random size/position/speed/alpha
33     for i in range(shapes_count):
34         var sprite := Sprite2D.new()
35         sprite.texture = texture
36         sprite.centered = true
37         var s := randf_range(min_scale, max_scale)
38         sprite.scale = Vector2(s, s)
39         sprite.position = Vector2(randf_range(-half_w, half_w), randf_range(-
half_h, half_h))
40         var a := randf_range(0.40, 0.72)
41         sprite.modulate = Color(1.0, 1.0, 1.0, a)
42         add_child(sprite)
43         sprites.append(sprite)
44         base_alphas.append(a)
45
46     # Fly in all directions
47     var vx: float = randf_range(-drift_x, drift_x)
48     var vy_abs: float = randf_range(min_speed_y, max_speed_y)
49     var vy: float = vy_abs if randf() < 0.5 else -vy_abs
50     speed.append(Vector2(vx, vy))
51     pulse_offsets.append(randf_range(0.0, TAU))

```

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ТПЖА.090301.957 ПЗ				Лист
									100
Изм	Лист	№ докум.	Подп.	Дата					

```

52
53 ## Moves, pulses and clones each frame
54 func _process(delta: float) -> void:
55     # Re-read viewport in case of resize
56     var viewport_size: Vector2 = get_viewport_rect().size
57     var half_w: float = viewport_size.x * 0.5
58     var half_h: float = viewport_size.y * 0.5
59
60     # Extra border outside screen
61     var wrap_margin: float = 60.0
62
63     for i in range.sprites.size():
64         var sprite := sprites[i]
65         # Move
66         sprite.position += speed[i] * delta
67
68         # Vertical wrap
69         if sprite.position.y > half_h + wrap_margin:
70             sprite.position.y = -half_h - wrap_margin
71             sprite.position.x = randf_range(-half_w, half_w)
72         elif sprite.position.y < -half_h - wrap_margin:
73             sprite.position.y = half_h + wrap_margin
74             sprite.position.x = randf_range(-half_w, half_w)
75
76         # Wrap horizontally
77         if sprite.position.x < -half_w - wrap_margin:
78             sprite.position.x = half_w + wrap_margin
79         elif sprite.position.x > half_w + wrap_margin:
80             sprite.position.x = -half_w - wrap_margin
81
82         # Alpha sin pulse
83         var pulse: float = sin((Time.get_ticks_msec() / 1000.0) * pulse_speed
+ pulse_offsets[i]) * pulse_strength
84         var target_alpha: float = clamp(base_alphas[i] + pulse, 0.20, 1.0)
85         sprite.modulate.a = target_alpha

```

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл.	

Изм.	Лист	№ докум.	Подп.	Дата
------	------	----------	-------	------

ТПЖА.090301.957 ПЗ

Лист
101

Листинг общего UI-компонента подсказок управления — controls_hint.gd

```
1 extends CanvasLayer
2
3 signal confirmed
4
5 @onready var ok_button: Button = $Panel/OkButton
6 var is_confirmed: bool = false
7
8 ## Bind controls hint actions
9 func _ready() -> void:
10     ok_button.pressed.connect(confirm_hint)
11
12 ## Emit signal once and close
13 func confirm_hint() -> void:
14     if (is_confirmed):
15         return
16     is_confirmed = true
17     confirmed.emit()
18     queue_free()
```

Подп. и дата	
Инов. № дубл.	
Взам. инв. №	
Подп. и дата	
Инов. № подл.	

Изм	Лист	№ докум.	Подп.	Дата

ТПЖА.090301.957 ПЗ